WTL →

Experiment 1 — Personal Profile Webpage Using AI-Assisted HTML Code Generation

Aim:

To design and develop a responsive personal profile webpage using AI-assisted HTML code generation, HTML5, CSS3, and JavaScript to showcase personal information, academic background, technical skills, projects, and contact details.

Objectives:

- To use AI tools to generate and improve webpage code.
- To structure the webpage using semantic HTML elements.
- To design an attractive and responsive interface using CSS.
- To implement animations and interactive features using JavaScript.
- To test the webpage across different screen sizes.

Theory:

A personal profile webpage presents an individual's background, skills, projects, and achievements in a structured digital format. HTML5 defines the webpage structure, CSS3 controls its appearance and responsiveness, and JavaScript adds interactive behaviour.

AI-assisted coding tools help generate code from natural-language prompts, explain programming concepts, and suggest improvements. However, the generated code must be reviewed and tested before use.

A responsive design adapts to different screen sizes using CSS Grid, Flexbox, and media queries. JavaScript can provide animations, scroll-based effects, and dynamic text.

Software / Tools Used

- HTML5, CSS3, JavaScript
- AI coding assistant
- Visual Studio Code
- Google Chrome or Microsoft Edge

Source Code :

```
<!DOCTYPE html>
<html>
<head>
<title>Pankaj Rana | Portfolio</title>
<meta name="viewport" content="width=device-width,initial-scale=1">
<style>
*{box-sizing:border-box;margin:0;padding:0}
body{font-family:Arial,sans-serif;background:#f5f8ff;color:#172033;
height:100vh;display:grid;place-items:center}
.box{width:92%;max-width:1050px;background:white;padding:38px;
border-radius:28px;box-shadow:0 20px 60px #b8c5dc55;border:1px solid #e6ebf5}
.top{display:flex;justify-content:space-between;align-items:center}
h1{font-size:52px;color:#172033}.name{color:#5b6cff}
.role{color:#68738a;font-size:18px;margin:7px 0 14px}
.bio{color:#667085;max-width:650px}
.contact{color:#5b6cff;margin-top:15px;font-size:14px}
.avatar{font-size:70px;background:#eef1ff;padding:22px;border-radius:50%}
.grid{display:grid;grid-template-columns:repeat(3,1fr);gap:15px;margin-top:28px}
.card{background:#f8faff;border:1px solid #e5eafa;padding:20px;border-radius:17px;
transition:.3s}.card:hover{transform:translateY(-5px);box-shadow:0 12px 25px #c5cce055}
h2{font-size:17px;color:#5b6cff;margin-bottom:9px}
p{font-size:14px;color:#68738a;line-height:1.5}
.skills{display:flex;gap:8px;flex-wrap:wrap;margin-top:20px}
.tag{background:#eef1ff;color:#4f5edb;padding:7px 12px;border-radius:20px;font-size:12px}
.links{margin-top:22px;display:flex;gap:10px}
.btn{background:#5b6cff;color:white;padding:10px 16px;border-radius:9px;
text-decoration:none;font-size:13px}.btn.light{background:#f0f2f8;color:#30394d}
@media(max-width:700px){body{height:auto;padding:20px}.top{display:block}
.avatar{display:none}.grid{grid-template-columns:1fr}h1{font-size:40px}}
</style>
</head>

<body>
<div class="box">

<div class="top">
<div>
<h1>Pankaj <span class="name">Rana</span></h1>
<div class="role">AI/ML Engineer in Progress ✨</div>
<p class="bio">B.Tech Computer Engineering student passionate about
AI, Machine Learning, Computer Vision and building useful software.</p>
<div class="contact">📍 Mumbai, Maharashtra   •   ✉ pankajrana2754@gmail.com</div>
</div>
<div class="avatar">🤖</div>
</div>
```

```
<div class="avatar">🤖</div>
</div>

<div class="grid">
<div class="card"><h2>🎓 Education</h2>
<p><b>B.Tech Computer Engineering</b><br>
St. John College of Engineering & Management<br>2024 - 2028</p></div>

<div class="card"><h2>💼 Experience</h2>
<p><b>WISERLI Internship</b><br>
Computer Vision • YOLOv8<br>Training • Evaluation • Deployment</p></div>

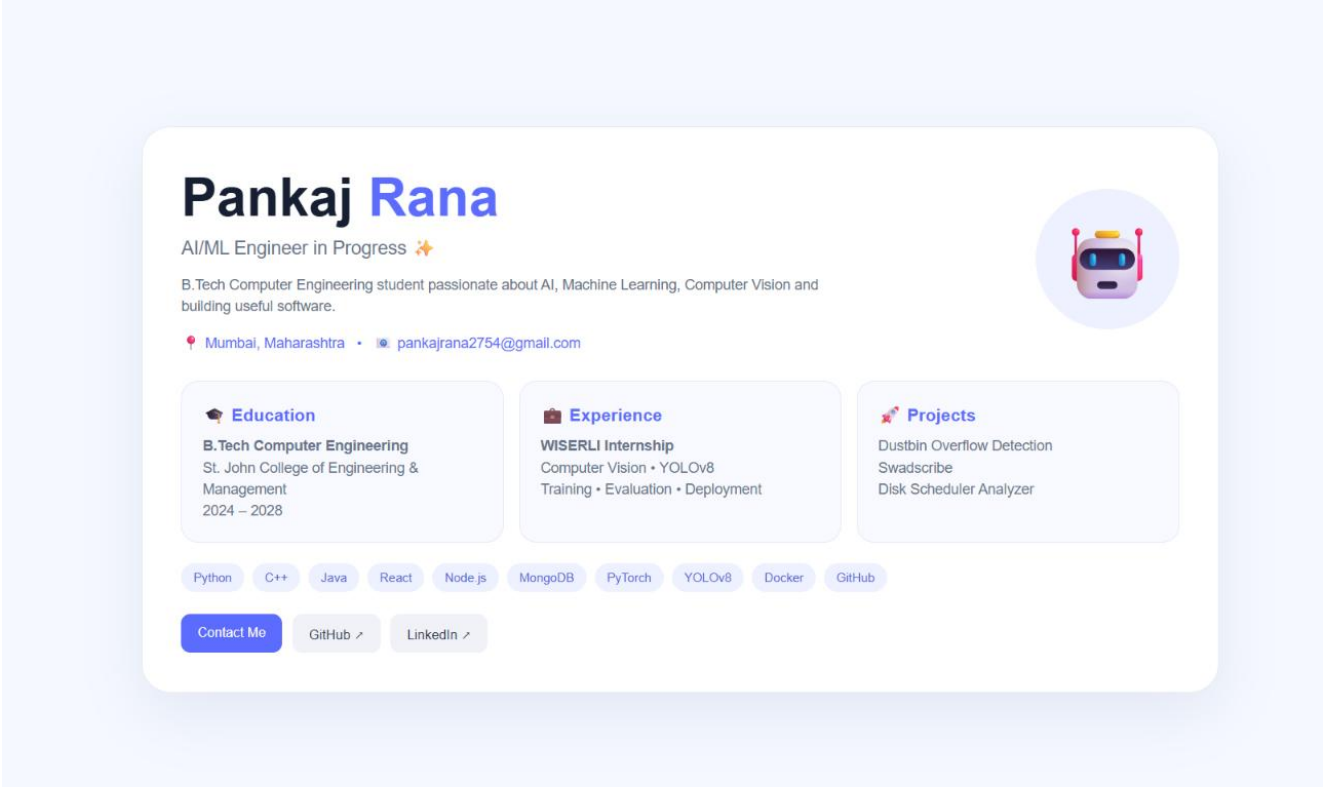
<div class="card"><h2>🚀 Projects</h2>
<p>Dustbin Overflow Detection<br>
Swadscribe<br>
Disk Scheduler Analyzer</p></div>
</div>
```

```
<div class="skills">
<span class="tag">Python</span><span class="tag">C++</span><span class="tag">Java</span>
<span class="tag">React</span><span class="tag">Node.js</span>
<span class="tag">MongoDB</span><span class="tag">PyTorch</span>
<span class="tag">YOLOv8</span><span class="tag">Docker</span><span class="tag">GitHub</span>
</div>

<div class="links">
<a class="btn" href="mailto:pankajrana2754@gmail.com">Contact Me</a>
<a class="btn light" href="https://github.com/Pankaj0536">GitHub ↗</a>
<a class="btn light" href="https://www.linkedin.com/">LinkedIn ↗</a>
</div>

</div>
</body>
</html>
```

Output



Experiment 2 — Multi-Page Responsive Website Using AI-Generated HTML and CSS

Aim:

To design and develop a multi-page responsive website using AI-generated HTML and CSS, with consistent navigation, page layouts, and styling across different devices.

Objectives:

- To generate webpage structures using AI prompts.
- To create multiple interconnected web pages.
- To implement consistent navigation between pages.
- To apply responsive CSS layouts.
- To validate navigation and layout behaviour in a browser.

Theory:

A multi-page website consists of several separate HTML pages connected through hyperlinks. Each page may serve a different purpose, such as Home, About, Services, and Contact.

HTML provides the structure and hyperlinks, while CSS controls layout, typography, colours, spacing, and responsive behaviour. Media queries allow styles to change according to screen width. Flexbox and CSS Grid help arrange elements efficiently.

AI-generated code can speed up the initial development process by producing HTML structures and CSS rules from written requirements. The developer must review the generated code for broken links, duplicate styles, accessibility issues, and responsiveness.

Software / Tools Used :

- HTML5
- CSS3
- AI coding assistant
- Visual Studio Code
- Web browser and developer tools

Source Code :

```
            const buttons = document.querySelectorAll('.menu-btn');
            buttons.forEach(btn => btn.classList.remove('active-menu'));
            element.classList.add('active-menu');
        }

        // Auto collapse menu on mobile viewports after pick
        if(window.innerWidth <= 800) {
            document.getElementById('sidebarPanel').classList.add('collapsed');
        }
    }

    function showAuthForm(mode) {
        const isLogin = mode === 'login';
        document.getElementById('login-form').hidden = !isLogin;
        document.getElementById('signup-form').hidden = isLogin;
        document.getElementById('login-tab').classList.toggle('active-tab', isLogin);
        document.getElementById('signup-tab').classList.toggle('active-tab', !isLogin);
        document.getElementById('login-tab').setAttribute('aria-selected', isLogin);
        document.getElementById('signup-tab').setAttribute('aria-selected', !isLogin);
        document.getElementById('auth-message').textContent = '';
    }

    function logIn(event) {
        event.preventDefault();
        document.getElementById('side-menu').style.display = 'block';
        document.getElementById('topNavBar').style.display = 'flex';
        navigateTo('dashboard-page');
        document.querySelector('.menu-btn').classList.add('active-menu');
    }

    function signUp(event) {
        event.preventDefault();
        const pin = document.getElementById('signup-pin').value;
        const confirmPin = document.getElementById('confirm-pin').value;
        if (pin !== confirmPin) {
            document.getElementById('auth-message').textContent = 'The PINs do not match. Please try again.';
            document.getElementById('confirm-pin').focus();
            return;
        }

        document.getElementById('acc-number').value = document.getElementById('signup-account').value;
        document.getElementById('pin').value = pin;
        document.getElementById('signup-form').reset();
        showAuthForm('login');
        document.getElementById('auth-message').textContent = 'Account request complete for this demo. You can now log in.';
    }

    function logOut() {
        document.getElementById('side-menu').style.display = 'none';
        document.getElementById('topNavBar').style.display = 'none';
        document.getElementById('acc-number').value = '';
        document.getElementById('pin').value = '';
        navigateTo('login-page');
    }
</script>
</body>
</html>
```

```
                <div class="timeline-container">
                    <div class="tx-row safe-in">
                        <div class="tx-left">
                            <span class="tx-badge">✔ SAFE</span>
                            <div class="tx-desc">Monthly Salary Received</div>
                            <div class="tx-time">July 15, 2026</div>
                        </div>
                        <div class="tx-right text-green">+$1,200.00</div>
                    </div>

                    <div class="tx-row safe-out">
                        <div class="tx-left">
                            <span class="tx-badge">✔ SAFE</span>
                            <div class="tx-desc">Grocery Store Bill Paid</div>
                            <div class="tx-time">July 14, 2026</div>
                        </div>
                        <div class="tx-right text-red">-$85.00</div>
                    </div>

                    <!-- FRAUD DETECTED FLAGGED ENTRY -->
                    <div class="tx-row fraud-warn">
                        <div class="tx-left">
                            <span class="tx-badge badge-alert">⚠ FRAUD DETECTED</span>
                            <div class="tx-desc">Unknown Store Spend - New York City</div>
                            <div class="tx-time">July 12, 2026 (Unusual Location Area)</div>
                        </div>
                        <div class="tx-right text-red">-$450.00</div>
                    </div>
                </div>
            </section>

        </main>
    </div>

    <!-- Layout Navigation & Interface Controls -->
    <script>
        function toggleSidebar() {
            const sidebar = document.getElementById('sidebarPanel');
            sidebar.classList.toggle('collapsed');
        }

        function navigateTo(pageId, element) {
            const pages = document.querySelectorAll('.page');
            pages.forEach(page => page.classList.remove('active-page'));

            const selectedPage = document.getElementById(pageId);
            if (selectedPage) {
                selectedPage.classList.add('active-page');
            }
```

```
            <div class="timeline-container">
                <div class="tx-row safe-in">
                    <div class="tx-left">
                        <span class="tx-badge">✔ SAFE</span>
                        <div class="tx-desc">Monthly Salary Received</div>
                        <div class="tx-time">July 15, 2026</div>
                    </div>
                    <div class="tx-right text-green">+$1,200.00</div>
                </div>

                <div class="tx-row safe-out">
                    <div class="tx-left">
                        <span class="tx-badge">✔ SAFE</span>
                        <div class="tx-desc">Grocery Store Bill Paid</div>
                        <div class="tx-time">July 14, 2026</div>
                    </div>
                    <div class="tx-right text-red">-$85.00</div>
                </div>

                <!-- FRAUD DETECTED FLAGGED ENTRY -->
                <div class="tx-row fraud-warn">
                    <div class="tx-left">
                        <span class="tx-badge badge-alert">⚠ FRAUD DETECTED</sp
                        <div class="tx-desc">Unknown Store Spend - New York City
                        <div class="tx-time">July 12, 2026 (Unusual Location Are
                    </div>
                    <div class="tx-right text-red">-$450.00</div>
                </div>
            </div>
```

```
        <!-- Status Card -->
        <div class="card status-card">
            <span class="card-icon">🛡️</span>
            <h3>Account Status</h3>
            <div class="status-badge">SAFE & ACTIVE ✔</div>
        </div>
    </div>

    <div class="helper-notice">
        <p>💡 Use the <b>Transaction Detection</b> menu option on the left side to look for suspicious or fake money activity.</p>
    </div>
</section>

<!-- ================ PAGE 3: DETAILS ================ -->
<section id="details-page" class="page">
    <div class="section-header">
        <h2>📄 Customer Profile Details</h2>
    </div>

    <div class="data-table">
        <div class="table-row">
            <span class="label">Full Name:</span>
            <span class="value">John Doe</span>
        </div>
        <div class="table-row">
            <span class="label">Account Number:</span>
            <span class="value">1234 - 5678 - 9000</span>
        </div>
        <div class="table-row">
            <span class="label">Registered Phone:</span>
            <span class="value">+1 (555) 019-2834</span>
        </div>
    </div>
</section>

<!-- ================ PAGE 4: TRANSACTIONS & FRAUD ================ -->
<section id="transactions-page" class="page">
    <div class="section-header">
        <h2>🔍 Fraud & Transaction Detection</h2>
    </div>

    <div class="color-guide">
        <div class="guide-item"><span class="indicator green-box"></span> Money In (Safe)</div>
        <div class="guide-item"><span class="indicator red-box"></span> Money Out (Safe)</div>
        <div class="guide-item"><span class="indicator yellow-box"></span> Warning (Suspicious Fraud!)</div>
    </div>

    <div class="timeline-container">
        <div class="tx-row safe-in">
            <div class="tx-left">
                <span class="tx-badge">✔ SAFE</span>
                <div class="tx-desc">Monthly Salary Received</div>
                <div class="tx-time">July 15, 2026</div>
            </div>
```

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Bank of India - Security Dashboard</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>

    <!-- ================ WEB LAYOUT CONTAINER ================ -->
    <div class="web-layout">

        <!-- SIDEBAR PANEL (Collapsible Branding & Menu) -->
        <aside class="sidebar" id="sidebarPanel">
            <!-- Close Button (Visible mainly on open states) -->
            <button class="close-sidebar-btn" onclick="toggleSidebar()">❌ Close Menu</button>

            <!-- Bank Logo & Name Box -->
            <div class="bank-logo-box">
                <div class="logo-circle">
                    <span class="logo-icon">🏛️</span>
                </div>
                <h1 class="bank-name">Bank of India</h1>
            </div>

            <!-- Navigation Links (Hidden during login, shown after) -->
            <nav class="sidebar-menu" id="side-menu">
                <button class="menu-btn active-menu" onclick="navigateTo('dashboard-page', this)">
                    <span class="menu-icon">🏠</span> Home
                </button>
                <button class="menu-btn" onclick="navigateTo('details-page', this)">
                    <span class="menu-icon">📄</span> Customer Details
                </button>
                <button class="menu-btn" onclick="navigateTo('transactions-page', this)">
                    <span class="menu-icon">🔍</span> Transaction Detection
                </button>
                <button class="menu-btn logout-btn" onclick="logOut()">
                    <span class="menu-icon">🚪</span> Exit / Log Out
                </button>
            </nav>
        </aside>

        <!-- MAIN CONTENT DISPLAY AREA -->
        <main class="main-content">

            <!-- Top bar to open menu if closed -->
            <div class="top-nav-bar" id="topNavBar" style="display: none;">
                <button class="open-sidebar-btn" onclick="toggleSidebar()">☰ Open Menu</button>
```

```
<div class="avatar">🤖</div>
</div>

<div class="grid">
<div class="card"><h2>🎓 Education</h2>
<p><b>B.Tech Computer Engineering</b><br>
St. John College of Engineering & Management<br>2024 – 2028</p></div>

<div class="card"><h2>💼 Experience</h2>
<p><b>WISERLI Internship</b><br>
Computer Vision • YOLOv8<br>Training • Evaluation • Deployment</p></div>

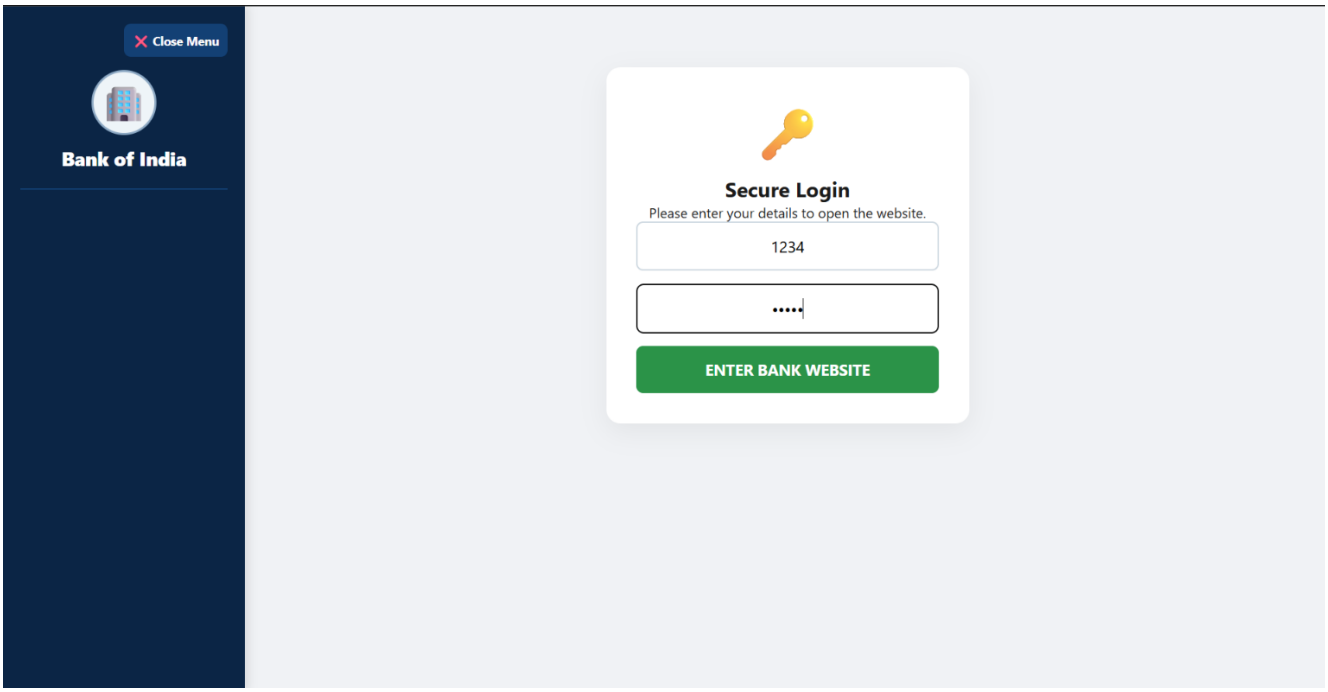
<div class="card"><h2>🚀 Projects</h2>
<p>Dustbin Overflow Detection<br>
Swadscribe<br>
Disk Scheduler Analyzer</p></div>
</div>

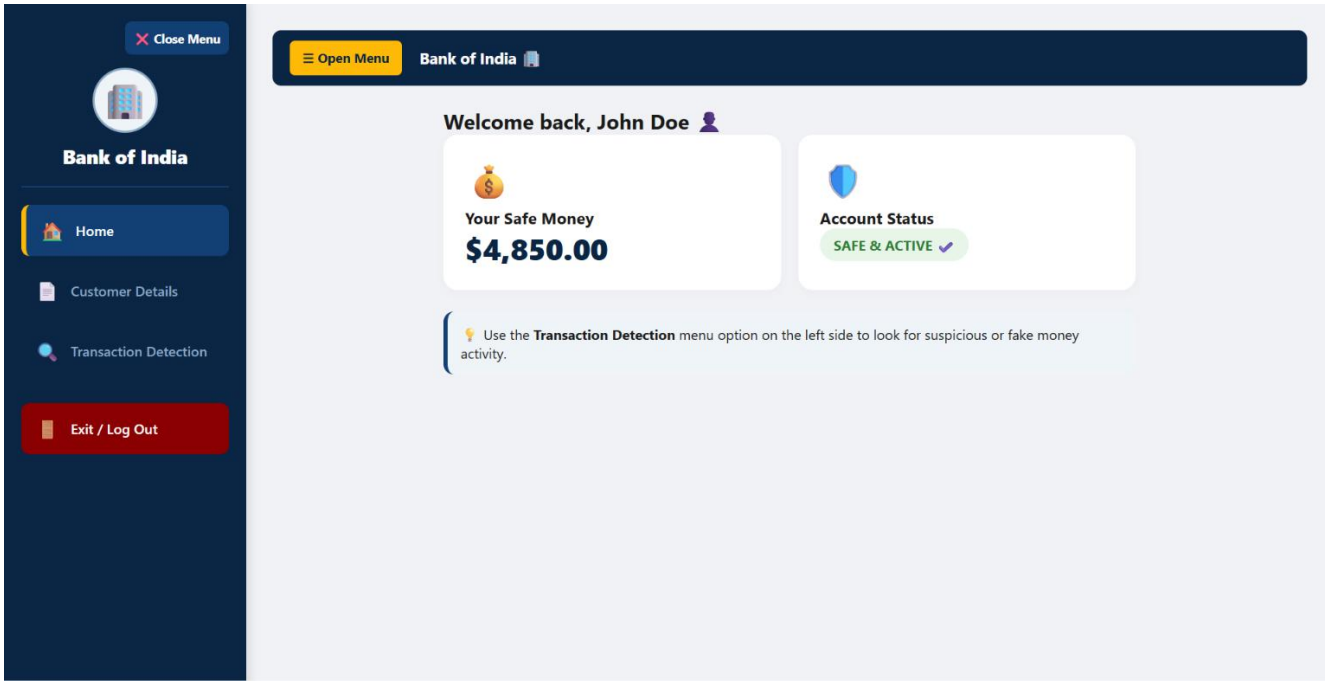
<div class="skills">
<span class="tag">Python</span><span class="tag">C++</span><span class="tag">Java</span>
<span class="tag">React</span><span class="tag">Node.js</span>
<span class="tag">MongoDB</span><span class="tag">PyTorch</span>
<span class="tag">YOLOv8</span><span class="tag">Docker</span><span class="tag">GitHub</span>
</div>

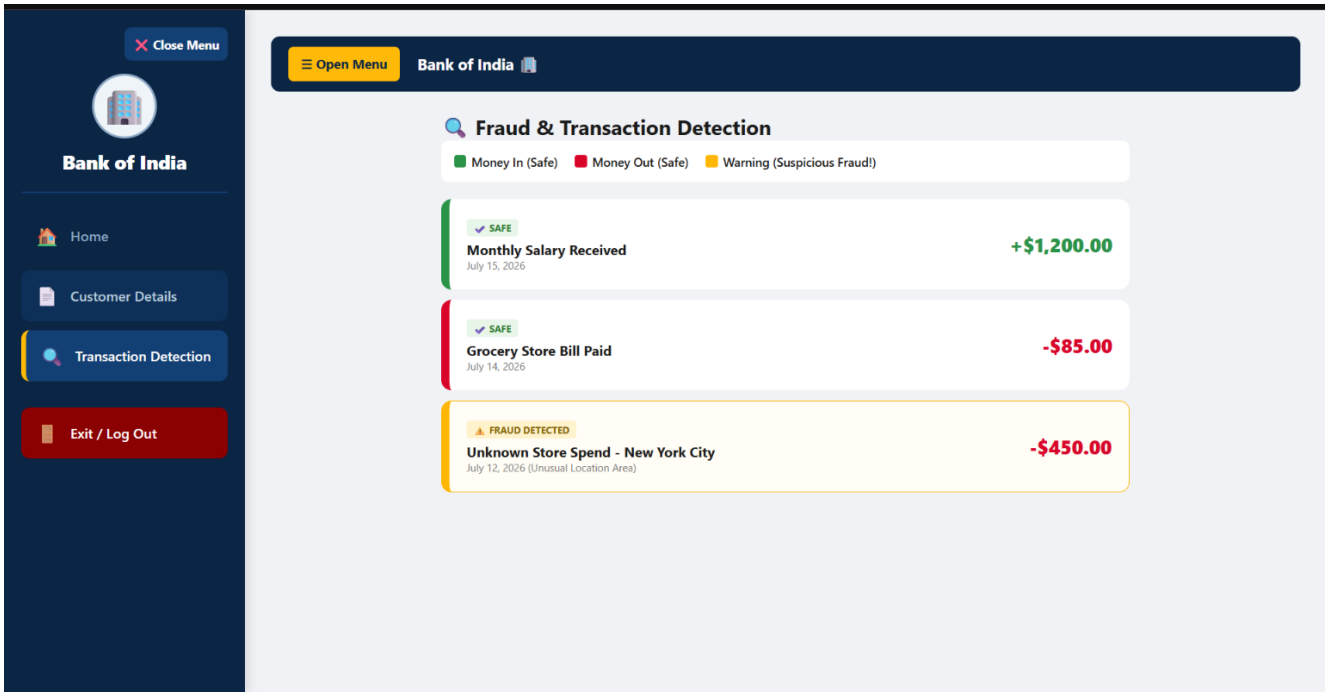
<div class="links">
<a class="btn" href="mailto:pankajrana2754@gmail.com">Contact Me</a>
<a class="btn light" href="https://github.com/Pankaj0536">GitHub ↗</a>
<a class="btn light" href="https://www.linkedin.com/">LinkedIn ↗</a>
</div>

</div>
</body>
</html>
```

Output :




Close Menu
Bank of India
Home
Customer Details
Transaction Detection
Exit / Log Out
Open Menu
Bank of India
Welcome back, John Doe
Your Safe Money
$4,850.00
Account Status
SAFE & ACTIVE
Use the Transaction Detection menu option on the left side to look for suspicious or fake money activity.


Close Menu
Bank of India
Home
Customer Details
Transaction Detection
Exit / Log Out
Open Menu
Bank of India
Fraud & Transaction Detection
Money In (Safe)
Money Out (Safe)
Warning (Suspicious Fraud!)
SAFE
Monthly Salary Received
July 15, 2026
+$1,200.00
SAFE
Grocery Store Bill Paid
July 14, 2026
-$85.00
FRAUD DETECTED
Unknown Store Spend - New York City
July 12, 2026 (Unusual Location Area)
-$450.00

Experiment 3 — Interactive To-Do List Using AI-Assisted JavaScript

Aim

To develop an interactive To-Do List application using AI-assisted JavaScript coding to create, display, complete, and delete tasks dynamically.

Objectives

- To generate and understand JavaScript code with AI assistance.
- To manipulate webpage elements using the DOM.
- To handle user interactions through event listeners.
- To update task status dynamically.
- To implement task persistence if supported by the application.

Theory

A To-Do List application helps users organise activities by maintaining a list of tasks. HTML defines the input field, buttons, and task list. CSS provides the visual design, while JavaScript handles task creation, completion, and deletion.

The Document Object Model (DOM) represents the webpage as objects that JavaScript can access and modify. Event listeners detect actions such as button clicks and form submissions.

An array can store task objects during execution. If local storage is implemented, tasks can also remain available after the browser is refreshed.

AI tools can generate event-handling functions, identify logic errors, and suggest improvements. The resulting code should be tested with empty inputs, duplicate tasks, and repeated user actions.

Software / Tools Used

- HTML5 and CSS3
- JavaScript
- AI coding assistant
- Visual Studio Code
- Web browser

Source Code

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <meta name="theme-color" content="#f4f6f1">
    <title>Daymark | To-Do List</title>
    <link rel="stylesheet" href="style.css">
    <script src="script.js" defer></script>
</head>
<body>
    <div class="app-shell">
        <header class="topbar">
            <a class="brand" href="#main" aria-label="Daymark home">
                <span class="brand-mark" aria-hidden="true">D</span>
                <span>daymark<span class="brand-period">.</span></span>
            </a>
            <div class="topbar-date" id="today-label"></div>
            <button class="button button-primary top-add" id="new-task-button" type="button">
                <span class="button-icon" aria-hidden="true">+</span>
                <span>New task</span>
            </button>
        </header>

        <div class="workspace">
            <aside class="sidebar" aria-label="Task navigation">
                <div class="sidebar-intro">
                    <p class="eyebrow">YOUR WORKSPACE</p>
                    <p class="sidebar-note">A little progress, every day.</p>
                </div>

                <nav class="filter-nav" aria-label="Filter tasks">
                    <button class="filter-button is-selected" type="button" data-filter="all" aria-pressed="true">
                        <span class="filter-symbol" aria-hidden="true">◷</span>
                        <span>All tasks</span>
                        <span class="filter-count" id="all-count">0</span>
                    </button>
                    <button class="filter-button" type="button" data-filter="active" aria-pressed="false">
                        <span class="filter-symbol" aria-hidden="true">○</span>
                        <span>To do</span>
                        <span class="filter-count" id="active-count">0</span>
                    </button>
                    <button class="filter-button" type="button" data-filter="completed" aria-pressed="false">
                        <span class="filter-symbol" aria-hidden="true">✓</span>
                        <span>Completed</span>
                        <span class="filter-count" id="completed-count">0</span>
                    </button>
                </nav>

                <section class="progress-panel" aria-labelledby="progress-title">
                    <div class="progress-heading">
                        <h2 id="progress-title">Daily progress</h2>
                        <span id="progress-percent">0%</span>
                    </div>
                    <div class="progress-track" role="progressbar" aria-label="Tasks completed" aria-valuemin="0" aria-valuemax="100" aria-valuenow="0" id="progress-bar">
```

```
            <label class="field-label" for="task-title">Task</label>
            <input class="form-control" id="task-title" name="title" type="text" maxlength="120" placeholder="What needs to get done?" required autocomplete="off">

            <div class="form-row">
                <div class="form-field">
                    <label class="field-label" for="task-priority">Priority</label>
                    <select class="form-control" id="task-priority" name="priority">
                        <option value="normal">Normal</option>
                        <option value="high">High</option>
                        <option value="low">Low</option>
                    </select>
                </div>
                <div class="form-field">
                    <label class="field-label" for="task-due-date">Due date</label>
                    <div class="date-control">
                        <input class="form-control" id="task-due-date" name="dueDate" type="date">
                        <button class="button button-secondary today-button" id="set-today-button" type="button">Today</button>
                    </div>
                </div>
            </div>

            <div class="dialog-actions">
                <button class="button button-secondary" id="cancel-dialog" type="button">Cancel</button>
                <button class="button button-primary" id="save-task-button" type="submit">Add task</button>
            </div>
        </form>
    </dialog>
</body>
</html>
```

```
                    <span class="progress-fill" id="progress-fill"></span>
                </div>
                <p id="progress-caption">A fresh start. Pick one thing to begin.</p>
            </section>

            <div class="sidebar-footer">
                <span class="status-dot" aria-hidden="true"></span>
                <span>Saved on this device</span>
            </div>
        </aside>

        <main class="main-panel" id="main">
            <div class="main-heading">
                <div>
                    <p class="eyebrow" id="view-eyebrow">YOUR LIST</p>
                    <h1 id="view-title">All tasks</h1>
                    <p class="task-summary" id="task-summary">Make room for what matters.</p>
                </div>
                <button class="button button-secondary clear-button" id="clear-completed" type="button" hidden>Clear completed</button>
            </div>

            <div class="list-toolbar">
                <label class="search-box" for="task-search">
                    <svg viewBox="0 0 24 24" aria-hidden="true"><circle cx="10.8" cy="10.8" r="6.8"></circle><path d="m16 16 5 5"></path></svg>
                    <input id="task-search" type="search" placeholder="Search your tasks" autocomplete="off">
                </label>
                <span class="list-date" id="list-date"></span>
            </div>

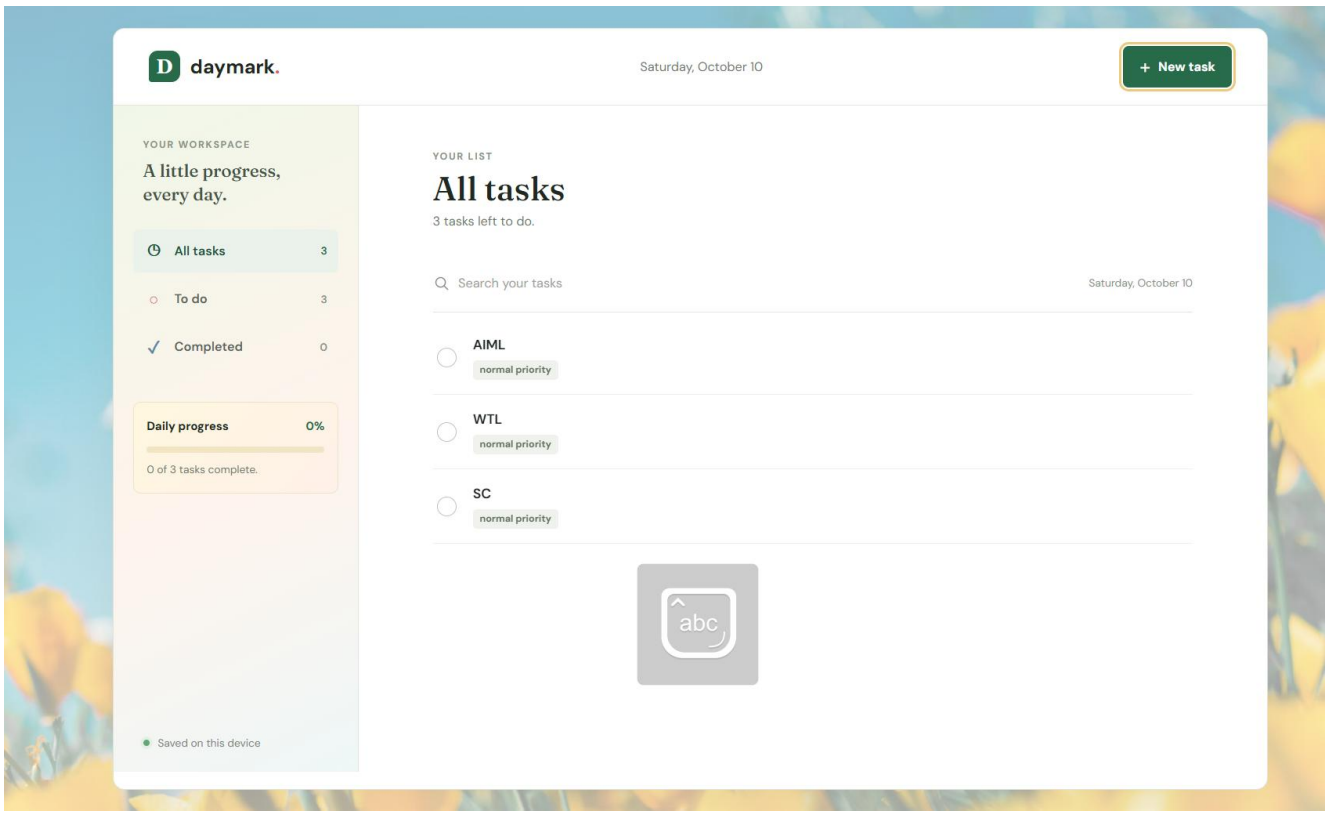
            <ul class="task-list" id="task-list" aria-label="Tasks"></ul>
            <div class="empty-state" id="empty-state" hidden>
                <div class="empty-mark" aria-hidden="true">✳</div>
                <h2 id="empty-title">Nothing on your list yet</h2>
                <p id="empty-description">Start with one small task. You can always add the next later.</p>
                <button class="button button-primary" id="empty-add-button" type="button">Create your first task</button>
            </div>
            <p class="sr-only" id="live-message" role="status" aria-live="polite"></p>
        </main>
    </div>
</div>

<dialog class="task-dialog" id="task-dialog" aria-labelledby="dialog-title">
    <form id="task-form" method="dialog">
        <div class="dialog-heading">
            <div>
                <p class="eyebrow">MAKE IT HAPPEN</p>
                <h2 id="dialog-title">New task</h2>
            </div>
            <button class="icon-button dialog-close" id="close-dialog" type="button" aria-label="Close dialog" title="Close">×</button>
        </div>

        <label class="field-label" for="task-title">Task</label>
        <input class="form-control" id="task-title" name="title" type="text" maxlength="120" placeholder="What needs to get done?" required autocomplete="off">

        <div class="form-row">
```

Output



Experiment 4 — Website Chatbot Using AI Services and Conversational Interfaces

Aim

To develop a website chatbot using AI services and a conversational interface that accepts user messages and provides relevant responses.

Objectives

- To understand the basic architecture of a chatbot.
- To design a user-friendly chat interface.
- To process user input and display responses.
- To integrate an AI service or response mechanism.
- To handle errors and invalid inputs appropriately.

Theory

A chatbot is a software application that interacts with users through text or other conversational interfaces. A website chatbot typically includes a chat window, message input, send button, and an area for displaying the conversation.

A rule-based chatbot selects responses using predefined conditions. An AI-powered chatbot may use a language model or an external conversational service to interpret user messages and generate responses.

A typical workflow consists of collecting the message, validating the input, sending it to the response engine, receiving the response, and displaying it in the chat window. When an external AI API is used, network errors, authentication, response time, and usage limits must be considered.

API credentials should be stored securely on a backend rather than exposed in client-side JavaScript.

Software / Tools Used

- HTML5 and CSS3
- JavaScript
- AI service or chatbot API, if integrated
- Visual Studio Code
- Web browser

Source Code

```
<div id="chatBox"></div>

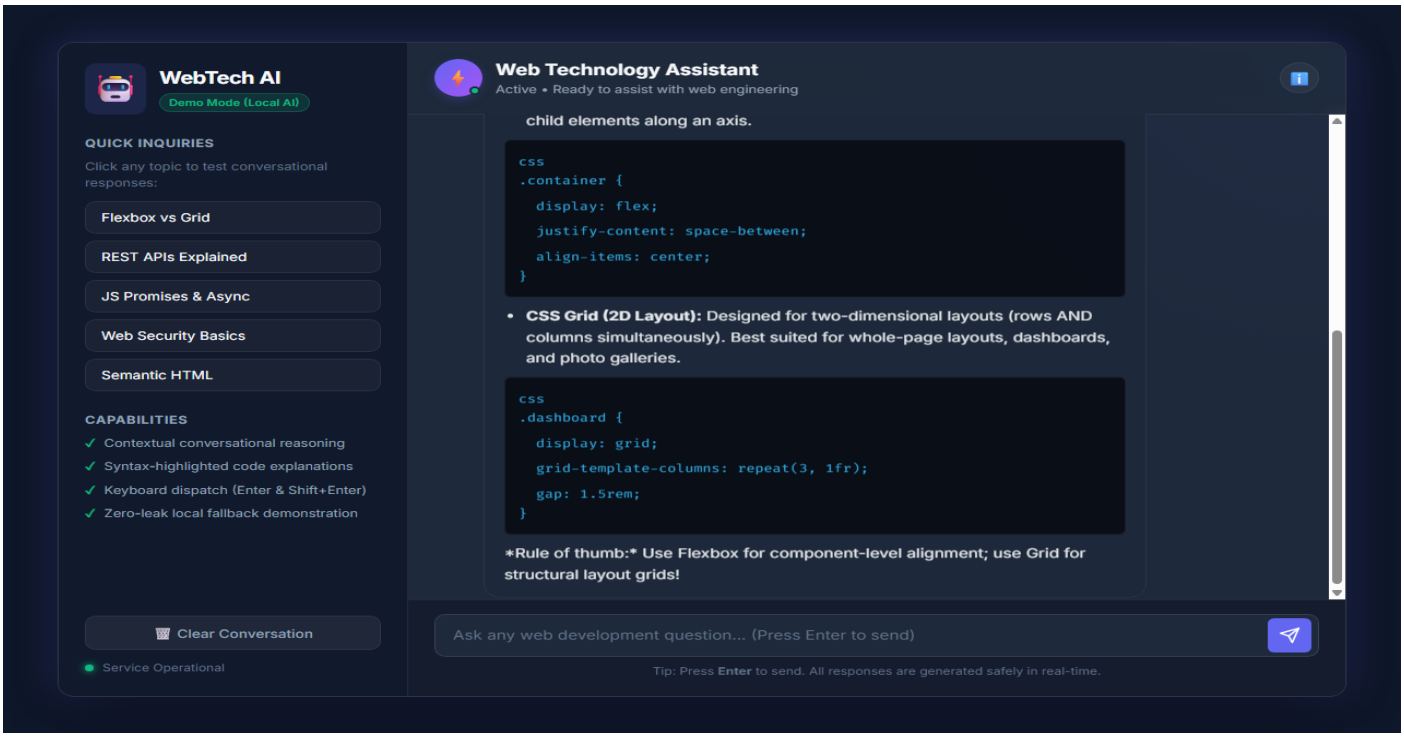

<input id="messageInput"
       placeholder="Type your message">

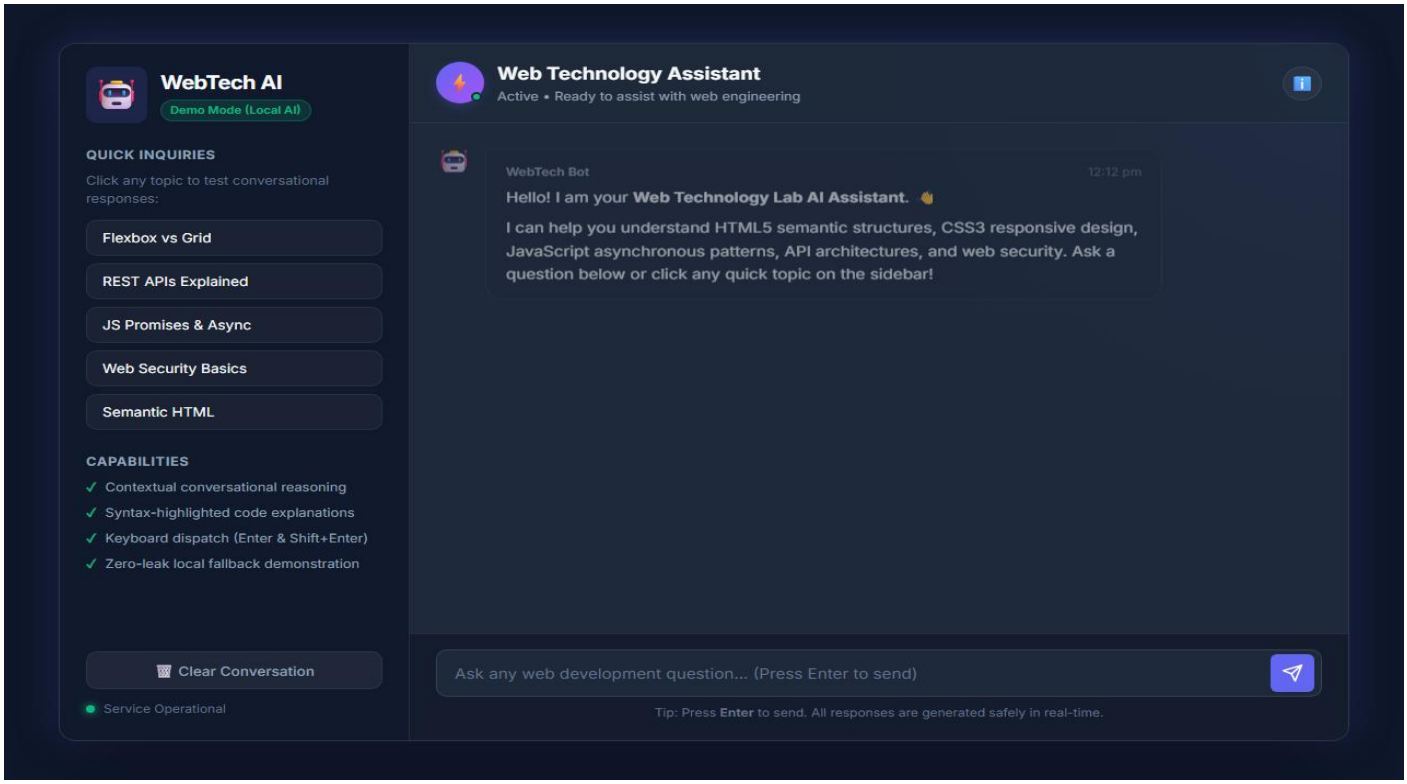

<button id="sendButton">Send</button>
const input = document.getElementById("messageInput");
const chatBox = document.getElementById("chatBox");


document.getElementById("sendButton")
    .addEventListener("click", function () {
        const message = input.value.trim();


        if (message !== "") {
            const userMessage = document.createElement("p");
            userMessage.textContent = "You: " + message;
            chatBox.appendChild(userMessage);
            input.value = "";
        }
    });
```

Output:





Conclusion:

The chatbot experiment demonstrates how conversational interfaces can be incorporated into websites. AI services can enable more flexible responses, while input validation and error handling improve reliability.

Experiment 5 — Identifying, Debugging, and Optimizing Errors Using AI Tools

Aim

To identify, debug, and optimize errors in an existing web project using AI tools, improving the correctness, performance, and maintainability of the application.

Objectives

- To identify syntax, runtime, and logical errors.
- To use AI tools to analyse error messages and suggest fixes.
- To understand and verify the proposed corrections.
- To optimize code readability and performance.
- To test the application after debugging.

Theory

Debugging is the process of locating and correcting defects in a program. Web projects may contain HTML structural issues, CSS layout problems, JavaScript exceptions, broken links, or inefficient code.

Browser developer tools provide a Console for viewing errors, an Elements panel for inspecting HTML and CSS, and a Network panel for identifying failed requests. AI coding assistants can analyse error messages, explain their causes, suggest corrected code, and recommend optimization techniques.

A systematic debugging process involves reproducing the problem, identifying the relevant code, applying a correction, and testing the application again. AI-generated fixes should not be accepted blindly because a suggested correction may introduce new problems.

Software / Tools Used

- Existing web project
- HTML, CSS, and JavaScript
- AI coding assistant
- Visual Studio Code
- Browser developer tools

Source Code

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Bank of India - Security Dashboard</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>

    <!-- ================== WEB LAYOUT CONTAINER ================== -->
    <div class="web-layout">

        <!-- SIDEBAR PANEL (Collapsible Branding & Menu) -->
        <aside class="sidebar" id="sidebarPanel">
            <!-- Close Button (Visible mainly on open states) -->
            <button class="close-sidebar-btn" onclick="toggleSidebar()">❌ Close Menu</button>

            <!-- Bank Logo & Name Box -->
            <div class="bank-logo-box">
                <div class="logo-circle">
                    <span class="logo-icon">🏢</span>
                </div>
                <h1 class="bank-name">Bank of India</h1>
            </div>

            <!-- Navigation Links (Hidden during login, shown after) -->
            <nav class="sidebar-menu" id="side-menu">
                <button class="menu-btn active-menu" onclick="navigateTo('dashboard-page', this)">
                    <span class="menu-icon">🏠</span> Home
```

```
        </button>
        <button class="menu-btn" onclick="navigateTo('details-page', this)">
            <span class="menu-icon">📄</span> Customer Details
        </button>
        <button class="menu-btn" onclick="navigateTo('transactions-page', this)">
            <span class="menu-icon">🔍</span> Transaction Detection
        </button>
        <button class="menu-btn logout-btn" onclick="logOut()">
            <span class="menu-icon">🚪</span> Exit / Log Out
        </button>
    </nav>
</aside>

<!-- MAIN CONTENT DISPLAY AREA -->
<main class="main-content">

    <!-- Top bar to open menu if closed -->
    <div class="top-nav-bar" id="topNavBar" style="display: none;">
        <button class="open-sidebar-btn" onclick="toggleSidebar()">☰ Open Menu</button>
        <span class="top-bar-title">Bank of India 🏦</span>
    </div>

    <!-- ================== PAGE 1: LOGIN ================== -->
    <section id="login-page" class="page active-page">
        <div class="login-window">
            <span class="auth-eyebrow">BANK OF INDIA</span>
            <h2>Welcome back</h2>
            <p class="helper-text">Sign in to view your account and recent activity.</p>
            <div class="auth-tabs" role="tablist" aria-label="Account access">
                <button class="auth-tab active-tab" id="login-tab" type="button" role="tab" aria-
selected="true" aria-controls="login-form" onclick="showAuthForm('login')">Log in</button>
```

```
            <button class="auth-tab" id="signup-tab" type="button" role="tab" aria-selected="false" aria-controls="signup-form" onclick="showAuthForm('signup')">Sign up</button>
          </div>
          <form id="login-form" class="auth-form" onsubmit="logIn(event)">
            <label for="acc-number">Account number</label>
            <input type="text" placeholder="e.g. 123456789000" id="acc-number" autocomplete="username" required />
            <label for="pin">PIN</label>
            <input type="password" placeholder="Enter your PIN" id="pin" autocomplete="current-password" required />
            <button class="btn btn-start" type="submit">Log in to your account</button>
          </form>
          <form id="signup-form" class="auth-form" onsubmit="signUp(event)" hidden>
            <label for="signup-name">Full name</label>
            <input type="text" placeholder="Enter your full name" id="signup-name" autocomplete="name" required />
            <label for="signup-account">Account number</label>
            <input type="text" placeholder="Enter your account number" id="signup-account" autocomplete="username" required />
            <label for="signup-pin">Create PIN</label>
            <input type="password" placeholder="At least 4 characters" id="signup-pin" autocomplete="new-password" minlength="4" required />
            <label for="confirm-pin">Confirm PIN</label>
            <input type="password" placeholder="Re-enter your PIN" id="confirm-pin" autocomplete="new-password" minlength="4" required />
            <button class="btn btn-start" type="submit">Create account</button>
          </form>
          <p class="auth-message" id="auth-message" role="status" aria-live="polite"></p>
          <p class="demo-note">Demo interface only. Do not enter real banking details.</p>
        </div>
      </section>
```

```
<!-- ================== PAGE 2: DASHBOARD ================== -->
<section id="dashboard-page" class="page">
    <div class="welcome-banner">
        <h2>Welcome back, <span class="user-highlight">John Doe</span> 👤 </h2>
    </div>

    <div class="dashboard-grid">
        <!-- Balance Card -->
        <div class="card balance-card">
            <span class="card-icon"> 💰 </span>
            <h3>Your Safe Money</h3>
            <div class="balance-amount">$4,850.00</div>
        </div>

        <!-- Status Card -->
        <div class="card status-card">
            <span class="card-icon"> 🛡️ </span>
            <h3>Account Status</h3>
            <div class="status-badge">SAFE & ACTIVE ✔ </div>
        </div>
    </div>

    <div class="helper-notice">
        <p> 💡 Use the <b>Transaction Detection</b> menu option on the left side to look for suspicious or fake money activity.</p>
    </div>
</section>

<!-- ================== PAGE 3: DETAILS ================== -->
<section id="details-page" class="page">
    <div class="section-header">
```

```
            <h2> 📄 Customer Profile Details</h2>
        </div>

        <div class="data-table">
            <div class="table-row">
                <span class="label">Full Name:</span>
                <span class="value">John Doe</span>
            </div>
            <div class="table-row">
                <span class="label">Account Number:</span>
                <span class="value">1234 - 5678 - 9000</span>
            </div>
            <div class="table-row">
                <span class="label">Registered Phone:</span>
                <span class="value">+1 (555) 019-2834</span>
            </div>
        </div>
    </section>

    <!-- ================== PAGE 4: TRANSACTIONS & FRAUD ================== -->
    <section id="transactions-page" class="page">
        <div class="section-header">
            <h2>🔍 Fraud & Transaction Detection</h2>
        </div>

        <div class="color-guide">
            <div class="guide-item"><span class="indicator green-box"></span> Money In (Safe)</div>
            <div class="guide-item"><span class="indicator red-box"></span> Money Out (Safe)</div>
```

```
            <div class="guide-item"><span class="indicator yellow-box"></span> Warning (Suspicious Fraud!)</div>
        </div>

        <div class="timeline-container">
            <div class="tx-row safe-in">
                <div class="tx-left">
                    <span class="tx-badge">✔ SAFE</span>
                    <div class="tx-desc">Monthly Salary Received</div>
                    <div class="tx-time">July 15, 2026</div>
                </div>
                <div class="tx-right text-green">+$1,200.00</div>
            </div>

            <div class="tx-row safe-out">
                <div class="tx-left">
                    <span class="tx-badge">✔ SAFE</span>
                    <div class="tx-desc">Grocery Store Bill Paid</div>
                    <div class="tx-time">July 14, 2026</div>
                </div>
                <div class="tx-right text-red">-$85.00</div>
            </div>

            <!-- FRAUD DETECTED FLAGGED ENTRY -->
            <div class="tx-row fraud-warn">
                <div class="tx-left">
                    <span class="tx-badge badge-alert">⚠ FRAUD DETECTED</span>
                    <div class="tx-desc">Unknown Store Spend - New York City</div>
                    <div class="tx-time">July 12, 2026 (Unusual Location Area)</div>
                </div>
                <div class="tx-right text-red">-$450.00</div>
```

```
            </div>
        </div>
    </section>

</main>
</div>

<!-- Layout Navigation & Interface Controls -->
<script>
    function toggleSidebar() {
        const sidebar = document.getElementById('sidebarPanel');
        sidebar.classList.toggle('collapsed');
    }

    function navigateTo(pageId, element) {
        const pages = document.querySelectorAll('.page');
        pages.forEach(page => page.classList.remove('active-page'));

        const selectedPage = document.getElementById(pageId);
        if (selectedPage) {
            selectedPage.classList.add('active-page');
        }

        if(element) {
            const buttons = document.querySelectorAll('.menu-btn');
            buttons.forEach(btn => btn.classList.remove('active-menu'));
            element.classList.add('active-menu');
        }

        // Auto collapse menu on mobile viewports after pick
        if(window.innerWidth <= 800) {
```

```
            document.getElementById('sidebarPanel').classList.add('collapsed');
        }
    }

    function showAuthForm(mode) {
        const isLogin = mode === 'login';
        document.getElementById('login-form').hidden = !isLogin;
        document.getElementById('signup-form').hidden = isLogin;
        document.getElementById('login-tab').classList.toggle('active-tab', isLogin);
        document.getElementById('signup-tab').classList.toggle('active-tab', !isLogin);
        document.getElementById('login-tab').setAttribute('aria-selected', isLogin);
        document.getElementById('signup-tab').setAttribute('aria-selected', !isLogin);
        document.getElementById('auth-message').textContent = '';
    }

    function logIn(event) {
        event.preventDefault();
        document.getElementById('side-menu').style.display = 'block';
        document.getElementById('topNavBar').style.display = 'flex';
        navigateTo('dashboard-page');
        document.querySelector('.menu-btn').classList.add('active-menu');
    }

    function signUp(event) {
        event.preventDefault();
        const pin = document.getElementById('signup-pin').value;
        const confirmPin = document.getElementById('confirm-pin').value;
        if (pin !== confirmPin) {
            document.getElementById('auth-message').textContent = 'The PINs do not match. Please try
again.';
            document.getElementById('confirm-pin').focus();
```

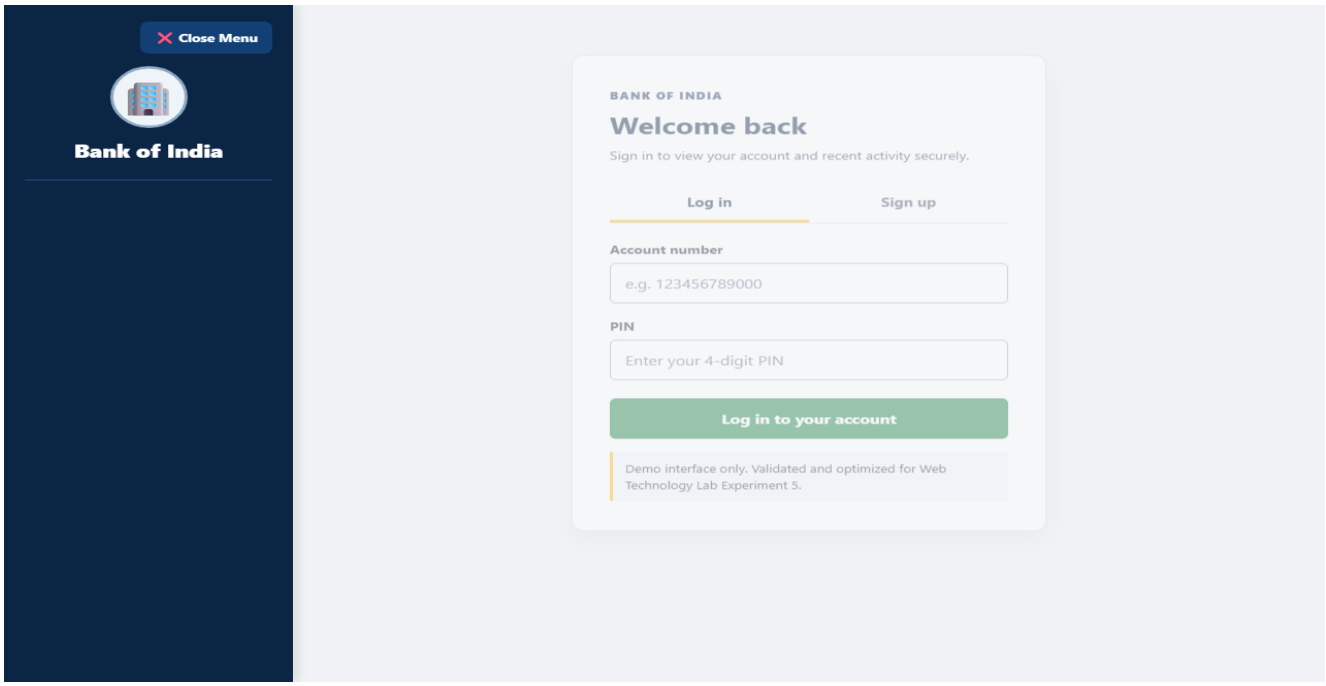
```
            return;
        }

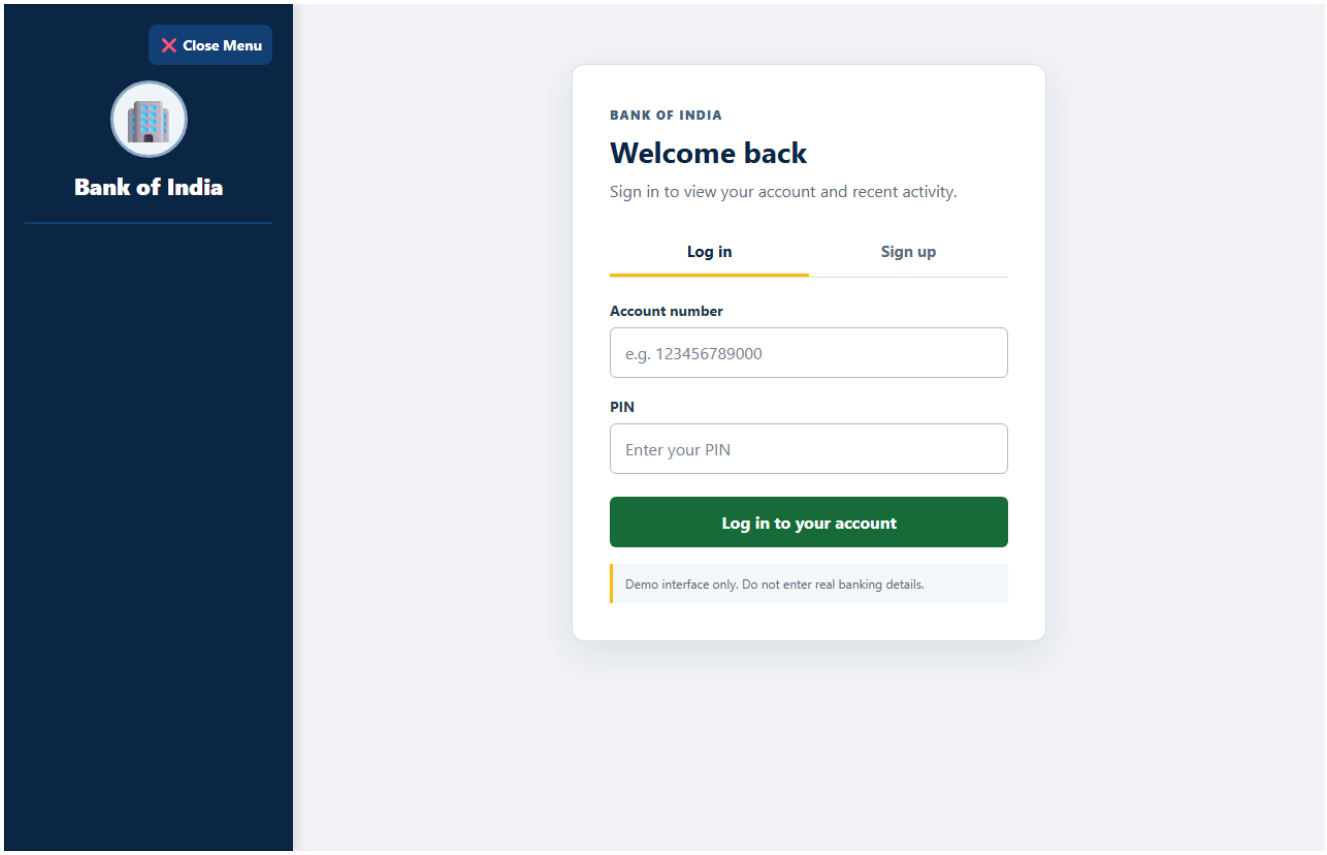
        document.getElementById('acc-number').value = document.getElementById('signup-account').value;
        document.getElementById('pin').value = pin;
        document.getElementById('signup-form').reset();
        showAuthForm('login');
        document.getElementById('auth-message').textContent = 'Account request complete for this demo. You can now log in.';
    }

    function logOut() {
        document.getElementById('side-menu').style.display = 'none';
        document.getElementById('topNavBar').style.display = 'none';
        document.getElementById('acc-number').value = '';
        document.getElementById('pin').value = '';
        navigateTo('login-page');
    }

    if (window.innerWidth <= 800) {
        document.getElementById('sidebarPanel').classList.add('collapsed');
    }
  </script>
</body>
</html>
```

Output





Conclusion

AI-assisted debugging can help developers understand errors and identify possible solutions quickly. Verification through execution and testing remains essential to ensure that the corrections work as intended.

Experiment 6 — Deploying an AI-Enabled Web Application on a Cloud Platform

Aim

To deploy an AI-enabled web application on a cloud platform and improve its performance, usability, and reliability using AI recommendations.

Objectives

- To understand cloud-based web application deployment.
- To prepare the application for production.
- To deploy the project using a suitable cloud platform.
- To verify the deployed application through its public URL.
- To evaluate AI-generated recommendations for optimization.

Theory

Cloud deployment makes a web application accessible over the internet without requiring every user to run the application locally. Cloud platforms provide hosting, build processes, deployment management, and, depending on the service, server-side execution and monitoring.

Before deployment, source files, dependencies, configuration settings, and build commands must be checked. A platform such as Vercel, Netlify, or another suitable hosting service may be used depending on the project's framework and requirements.

AI tools can recommend improvements to loading performance, responsive design, accessibility, code quality, and asset optimization. These recommendations should be evaluated through actual testing rather than assumed to improve performance.

Sensitive environment variables and API keys must be configured securely.

Software / Tools Used

- Existing AI-enabled web application
- Git and GitHub, if used
- Cloud hosting platform
- AI coding assistant
- Web browser and developer tools

Procedure

1. Review the application and its dependencies.
2. Prepare the production build, if required.
3. Connect the project repository or upload the required files.
4. Configure build settings and environment variables.
5. Deploy the application.

6. Open the deployed URL and test its functionality.
7. Review AI recommendations and apply suitable improvements.
8. Re-test the application after optimization.

Source Code :

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">

    <!-- SEO & Metadata Optimization (AI Recommendation) -->
    <title>WebTech AI Assistant - Production Cloud Edition</title>
    <meta name="description" content="AI-assisted Web Technology conversational assistant deployed for academic research and developer workflows.">
    <meta name="keywords" content="web technology, chatbot, ai assistant, frontend, javascript, html5, css3">
    <meta name="author" content="Third Year Computer Engineering - WTL">
    <meta name="theme-color" content="#0f172a">

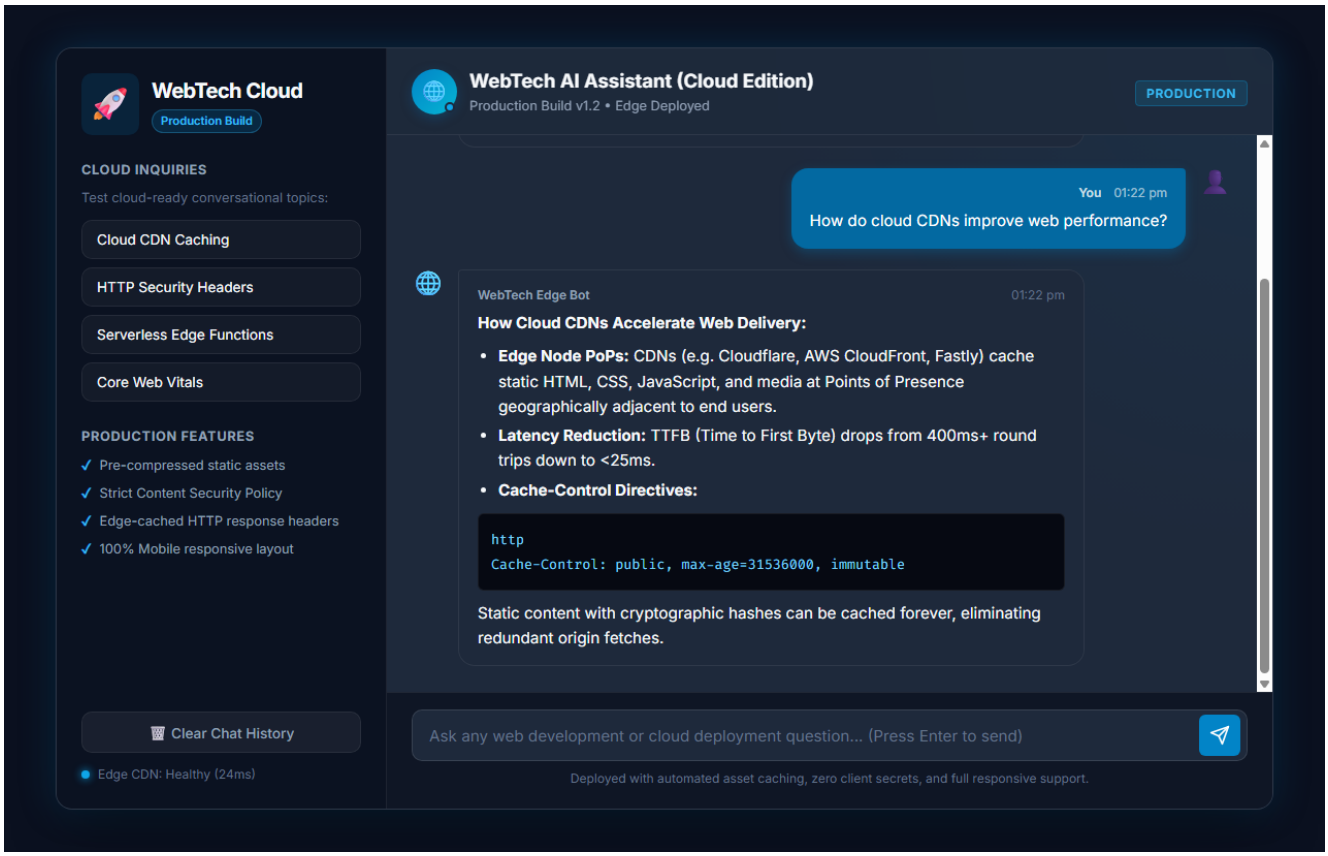
    <!-- Open Graph Protocol -->
    <meta property="og:title" content="WebTech AI Assistant">
    <meta property="og:description" content="Conversational AI agent for web engineering concepts and tutorials.">
    <meta property="og:type" content="website">

    <!-- Preconnect & Fonts -->
    <link rel="preconnect" href="https://fonts.googleapis.com">
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
    <link href="https://fonts.googleapis.com/css2?family=Inter:wght@400;500;600;700&family=Fira+Code:wght@400;500&display=swap" rel="stylesheet">

    <link rel="stylesheet" href="style.css">
</head>
<body>
    <div class="chat-app-container">
        <!-- Sidebar Panel -->
        <aside class="chat-sidebar" aria-label="Assistant Options">
            <div class="sidebar-header">
                <div class="app-logo">
                    <span class="logo-icon" aria-hidden="true">🚀</span>
                    <div class="logo-text">
                        <h2>WebTech Cloud</h2>
                        <span class="badge-mode">Production Build</span>
                    </div>
                </div>
            </div>

            <div class="sidebar-section">
                <h3>Cloud Inquiries</h3>
                <p class="section-desc">Test cloud-ready conversational topics:</p>
                <div class="suggestion-chips" id="suggestionChips">
                    <button class="chip" data-prompt="How do cloud CDNs improve web performance?">Cloud CDN Caching</button>
                    <button class="chip" data-prompt="What are standard HTTP security headers?">HTTP Security Headers</buttton>
                    <button class="chip" data-prompt="Explain Serverless Edge Functions vs Traditional Backends.">Serverless Edge Functions</button>
                    <button class="chip" data-prompt="What are essential Core Web Vitals metrics?">Core Web Vitals</button>
</div>
```

Output



Conclusion

Cloud deployment makes web applications accessible to users through the internet. AI recommendations can support optimization, but improvements must be verified through testing and suitable performance measurements.

Experiment 7 — Comparing Multiple AI Coding Assistants for Web Development

Aim

To compare multiple AI coding assistants for web development tasks and evaluate their effectiveness in generating correct, readable, and maintainable code.

Objectives

- To explore different AI coding assistants.
- To give comparable coding tasks to each tool.
- To evaluate the generated HTML, CSS, and JavaScript.
- To compare code correctness, responsiveness, and readability.
- To identify the strengths and limitations of each assistant.

Theory

AI coding assistants use language models and related techniques to generate, explain, complete, and modify source code from natural-language instructions or existing code.

Different tools may produce different implementations for the same task. Their output can vary in correctness, code structure, responsiveness, accessibility, explanation quality, and the amount of manual correction required.

A fair comparison requires the same task description, similar constraints, and a consistent evaluation process. The generated applications should be executed and tested rather than judged only by their appearance or the length of the code.

Possible evaluation criteria include:

- Functional correctness
- Code readability and maintainability
- Responsive design
- Accessibility
- Quality of explanations
- Time required for correction and testing

Software / Tools Used

- Two or more AI coding assistants
- HTML5, CSS3, and JavaScript
- Visual Studio Code
- Web browser
- A common test task and evaluation checklist

Procedure

1. Select at least two AI coding assistants.
2. Prepare one common web development task.
3. Submit the same prompt and requirements to each tool.
4. Save the generated source code.
5. Run each implementation in the browser.
6. Test functionality and responsive behaviour.
7. Compare the results using common criteria.

Source Code :

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Finance Tracker - Assistant A (Modular OOP)</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <div class="app-wrapper">
        <header class="app-header">
            <div class="header-content">
                <h1>💰 FinanceFlow Tracker</h1>
                <p class="subtitle">Implementation A: Modular Class-Based Architecture</p>
            </div>
            <span class="assistant-badge">Assistant A: GitHub Copilot Style</span>
        </header>

        <!-- KPI Metrics Grid -->
        <section class="metrics-grid" aria-label="Financial Summary Cards">
            <div class="metric-card balance-card">
                <span class="metric-label">Net Balance</span>
                <span class="metric-val" id="netBalance">$0.00</span>
            </div>
            <div class="metric-card income-card">
                <span class="metric-label">Total Income</span>
                <span class="metric-val text-green" id="totalIncome">+$0.00</span>
            </div>
            <div class="metric-card expense-card">
                <span class="metric-label">Total Expenses</span>
                <span class="metric-val text-red" id="totalExpense">-$0.00</span>
            </div>
        </section>

        <!-- Main Workspace -->
        <main class="main-layout">
            <!-- Add Transaction Section -->
            <section class="card form-section" aria-labelleddy="form-heading">
                <h2 id="form-heading">Add Transaction</h2>
                <form id="transactionForm">
                    <div class="form-group">
                        <label for="desc">Description</label>
                        <input type="text" id="desc" placeholder="e.g. Salary, Groceries, Internet" required />
                    </div>

                    <div class="form-row">
                        <div class="form-group">
                            <label for="amount">Amount ($)</label>
                            <input type="number" id="amount" step="0.01" min="0.01" placeholder="0.00" required />
                        </div>
                        <div class="form-group">
                            <label for="type">Type</label>
                            <select id="type" required>
                                <option value="income">Income (+)</option>
                                <option value="expense">Expense (-)</option>
                            </select>
                        </div>
                    </div>
                    <div class="form-group">
                        <label for="category">Category</label>
                        <select id="category" required>
                            <option value="Salary">Salary / Stipend</option>
                            <option value="Food">Food &amp; Dining</option>
                            <option value="Utilities">Utilities &amp; Bills</option>
                            <option value="Entertainment">Entertainment</option>
                            <option value="Education">Education &amp; Books</option>
                            <option value="Other">Other</option>
                        </select>
                    </div>

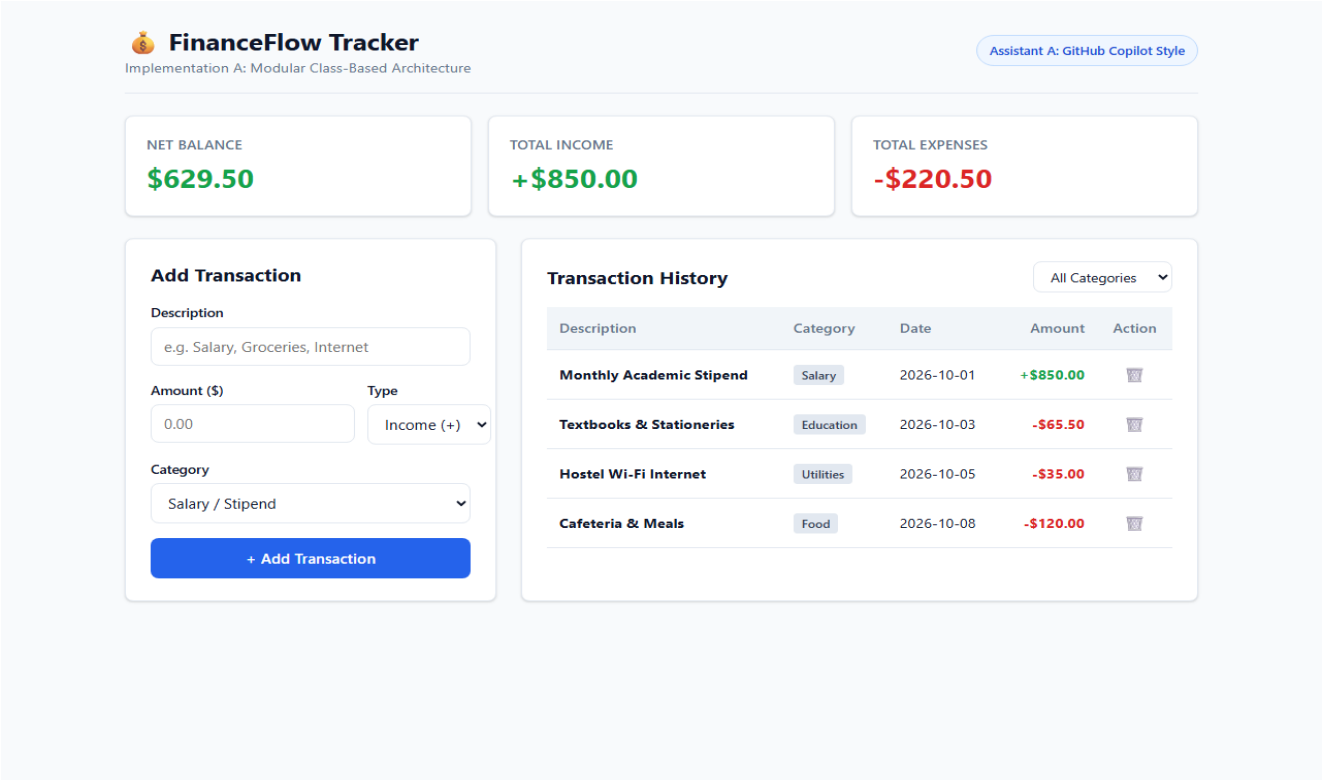
                    <button type="submit" class="btn btn-primary" id="addBtn">+ Add Transaction</button>
                </form>
            </section>

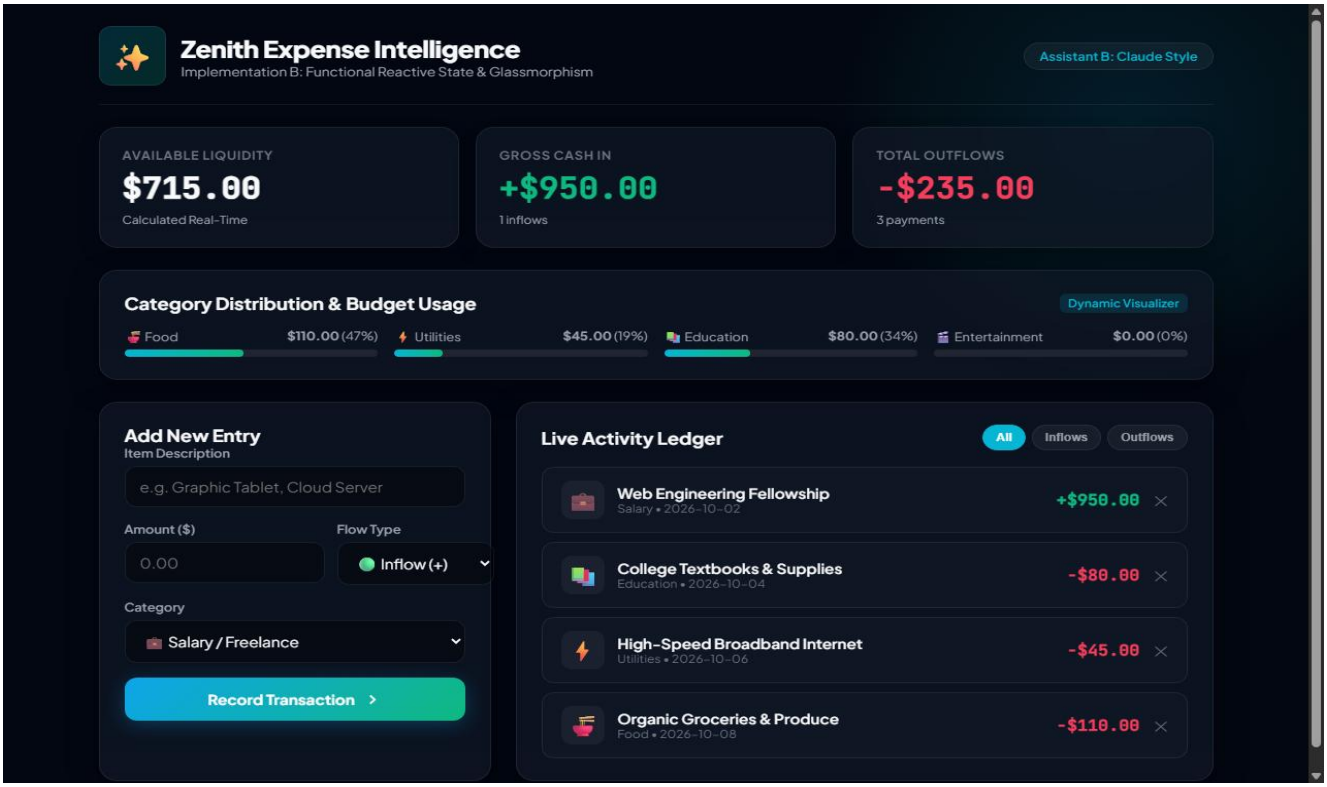
            <!-- Transactions List Section -->
            <section class="card list-section" aria-labelleddy="list-heading">
                <div class="list-header">
                    <h2 id="list-heading">Transaction History</h2>
                    <div class="filter-box">
                        <label for="categoryFilter" class="sr-only">Filter by category</label>
                        <select id="categoryFilter" aria-label="Filter transactions by category">
                            <option value="all">All Categories</option>
                            <option value="Salary">Salary</option>
                            <option value="Food">Food &amp; Dining</option>
                            <option value="Utilities">Utilities</option>
                            <option value="Entertainment">Entertainment</option>
                            <option value="Education">Education</option>
                            <option value="Other">Other</option>
                        </select>
                    </div>
                </div>

                <div class="transaction-table-wrapper">
                    <table class="transaction-table">
                        <thead>
                            <tr>
                                <th>Description</th>
                                <th>Category</th>
                                <th>Date</th>
                                <th class="text-right">Amount</th>
                                <th class="text-center">Action</th>
                            </tr>
                        </thead>
                        <tbody id="transactionBody">
                            <!-- Populated dynamically -->
                        </tbody>
                    </table>
                    <div class="empty-state hidden" id="emptyState">
                        <p>No transactions found for this filter.</p>
                    </div>
                </div>
            </section>
        </main>
    </div>

    <script src="script.js"></script>
</body>
</html>
```

Output





Conclusion

Comparing AI coding assistants helps developers understand the differences in their code-generation capabilities. The best tool should be selected according to the task, output quality, and amount of verification required.

Experiment – 8

1. Aim

To generate automated project documentation, test cases, and code comments using AI tools for the Multi-Page Responsive Website developed in the previous practical.

2. Objectives

- To generate project documentation using AI-assisted tools.
- To document the structure, features, and functionality of the multi-page responsive website.
- To create test cases for verifying webpage navigation, responsiveness, and functionality.
- To generate meaningful code comments to improve code readability and maintainability.
- To validate AI-generated documentation and test cases against the actual website implementation.

3. Theory

AI-Assisted Project Documentation

Project documentation provides information about the purpose, structure, features, installation, and usage of a software application. AI tools can analyse source code and generate documentation in formats such as Markdown and README files.

For a multi-page responsive website, documentation may include an overview of the project, technologies used, page descriptions, folder structure, and instructions for running the website.

Automated Test Case Generation

Test cases are used to verify whether an application behaves according to its requirements. AI tools can generate test scenarios based on the website's features and expected behaviour.

For a multi-page responsive website, test cases may cover navigation links, page loading, responsive layouts, images, forms, and browser compatibility. Each test case should specify the test scenario, expected result, and actual result.

AI-Generated Code Comments

Code comments explain the purpose and functionality of important sections of source code. AI tools can suggest comments for HTML structures, CSS layouts, and JavaScript functions.

For example, comments can explain the navigation menu, responsive media queries, and event-handling functions. Meaningful comments make the code easier to understand, debug, and maintain.

Role of AI Tools

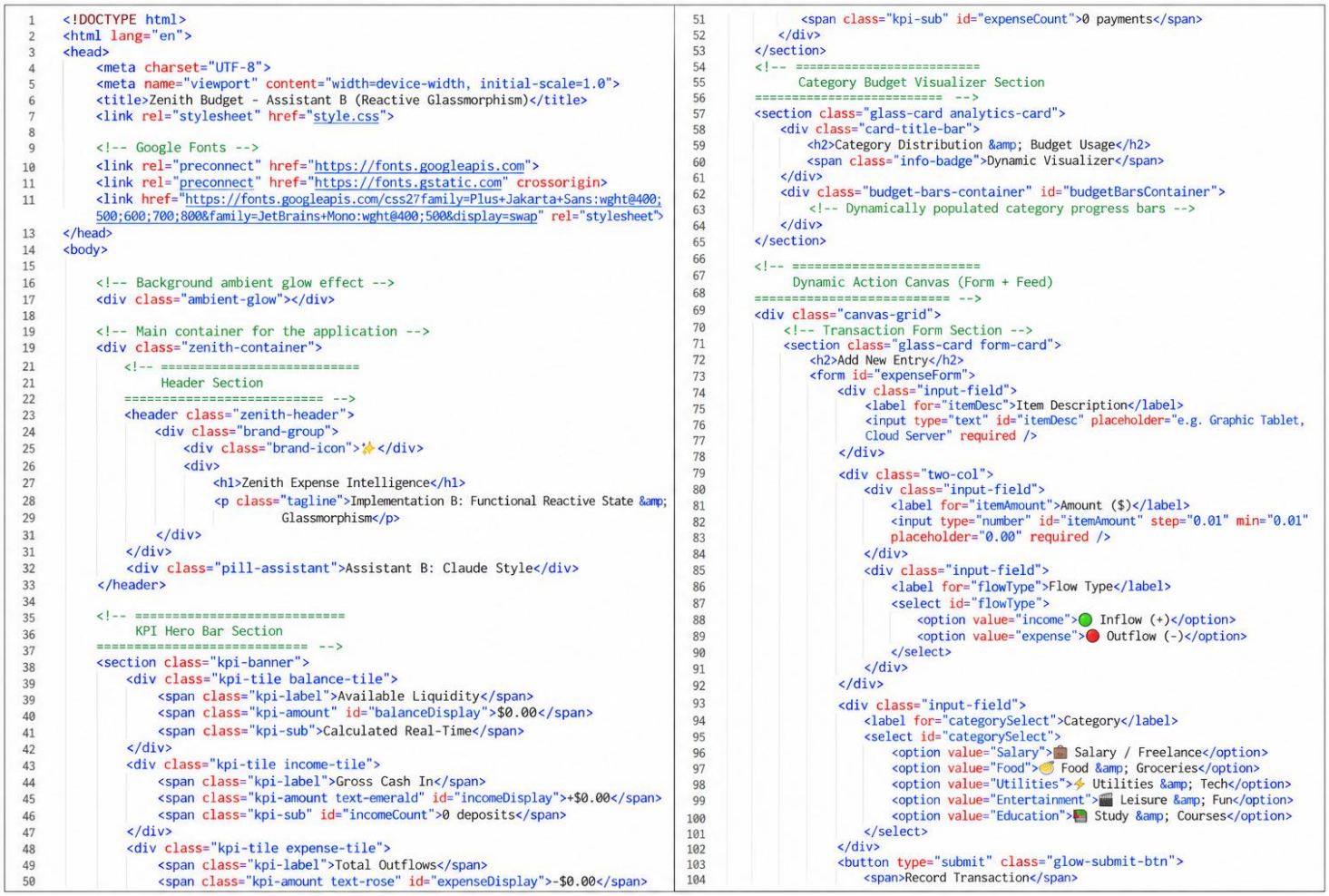
AI coding assistants help automate repetitive documentation tasks and suggest test cases and code comments. However, the generated content must be reviewed to ensure accuracy and consistency with the actual project. AI-generated test cases must also be executed to verify the website's behaviour.

4. Software / Tools Used

- HTML5

- CSS3
- JavaScript
- AI coding assistant
- Visual Studio Code
- Web browser and developer tools
- Markdown editor or README file

5. Source Code



```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Zenith Budget - Assistant B (Reactive Glassmorphism)</title>
    <link rel="stylesheet" href="style.css">

    <!-- Google Fonts -->
    <link rel="preconnect" href="https://fonts.googleapis.com">
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
    <link href="https://fonts.googleapis.com/css2?family=Plus+Jakarta+Sans:wght@400;500;600;700;800&family=JetBrains+Mono:wght@400;500&display=swap" rel="stylesheet">
</head>
<body>

    <!-- Background ambient glow effect -->
    <div class="ambient-glow"></div>

    <!-- Main container for the application -->
    <div class="zenith-container">
        <!-- ==========================
             Header Section
        ========================== -->
        <header class="zenith-header">
            <div class="brand-group">
                <div class="brand-icon">✨</div>
                <div>
                    <h1>Zenith Expense Intelligence</h1>
                    <p class="tagline">Implementation B: Functional Reactive State &amp;
                            Glassmorphism</p>
                </div>
            </div>
            <div class="pill-assistant">Assistant B: Claude Style</div>
        </header>

        <!-- ==========================
             KPI Hero Bar Section
        ========================== -->
        <section class="kpi-banner">
            <div class="kpi-tile balance-tile">
                <span class="kpi-label">Available Liquidity</span>
                <span class="kpi-amount" id="balanceDisplay">$0.00</span>
                <span class="kpi-sub">Calculated Real-Time</span>
            </div>
            <div class="kpi-tile income-tile">
                <span class="kpi-label">Gross Cash In</span>
                <span class="kpi-amount text-emerald" id="incomeDisplay">+$0.00</span>
                <span class="kpi-sub" id="incomeCount">0 deposits</span>
            </div>
            <div class="kpi-tile expense-tile">
                <span class="kpi-label">Total Outflows</span>
                <span class="kpi-amount text-rose" id="expenseDisplay">-$0.00</span>
                <span class="kpi-sub" id="expenseCount">0 payments</span>
            </div>
        </section>
        <!-- ==========================
             Category Budget Visualizer Section
        ========================== -->
        <section class="glass-card analytics-card">
            <div class="card-title-bar">
                <h2>Category Distribution &amp; Budget Usage</h2>
                <span class="info-badge">Dynamic Visualizer</span>
            </div>
            <div class="budget-bars-container" id="budgetBarsContainer">
                <!-- Dynamically populated category progress bars -->
            </div>
        </section>

        <!-- ==========================
             Dynamic Action Canvas (Form + Feed)
        ========================== -->
        <div class="canvas-grid">
            <!-- Transaction Form Section -->
            <section class="glass-card form-card">
                <h2>Add New Entry</h2>
                <form id="expenseForm">
                    <div class="input-field">
                        <label for="itemDesc">Item Description</label>
                        <input type="text" id="itemDesc" placeholder="e.g. Graphic Tablet, Cloud Server" required />
                    </div>

                    <div class="two-col">
                        <div class="input-field">
                            <label for="itemAmount">Amount ($)</label>
                            <input type="number" id="itemAmount" step="0.01" min="0.01" placeholder="0.00" required />
                        </div>
                        <div class="input-field">
                            <label for="flowType">Flow Type</label>
                            <select id="flowType">
                                <option value="income">🟢 Inflow (+)</option>
                                <option value="expense">🔴 Outflow (-)</option>
                            </select>
                        </div>
                    </div>

                    <div class="input-field">
                        <label for="categorySelect">Category</label>
                        <select id="categorySelect">
                            <option value="Salary">💼 Salary / Freelance</option>
                            <option value="Food">🍔 Food &amp; Groceries</option>
                            <option value="Utilities">⚡ Utilities &amp; Tech</option>
                            <option value="Entertainment">🎬 Leisure &amp; Fun</option>
                            <option value="Education">📚 Study &amp; Courses</option>
                        </select>
                    </div>
                    <button type="submit" class="glow-submit-btn">
                        <span>Record Transaction</span>
```

6. Output

Experiment 8: Commented Source Code

Automated Generation via AI Evaluation Pipeline • WTL Experiment 8

```
<!DOCTYPE html>
<!--
================================================================================
BANK OF INDIA - SECURITY DASHBOARD
Practical Lab Target: WTL Experiment 2 / Documented for Experiment 8
File: index.html (Commented Source Code Edition)
Author: Computer Engineering Dept (WTL Laboratory)
================================================================================
This application is a responsive, client-side Single Page Application (SPA)
emulating a multi-page banking security dashboard with:
1. Authentication (Login / Sign-up tabs)
2. Safe Money & Account Status Dashboard
3. Customer Profile Information
4. Financial Fraud & Anomalous Transaction Detection Timeline
================================================================================
-->
<html lang="en">
<head>
    <!-- Character encoding for standard alphanumeric and emoji characters -->
    <meta charset="UTF-8">
    <!-- Viewport definition ensuring 1:1 scale rendering across mobile and desktop -->
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Bank of India - Security Dashboard (Fully Commented)</title>
    <!-- External stylesheet defining layouts, colors, and transitions -->
    <link rel="stylesheet" href="style.css">
</head>
<body>

    <!-- ========================================================================
         PRIMARY LAYOUT WRAPPER: Flexbox container orchestrating sidebar & canvas
         ======================================================================== -->
    <div class="web-layout">
```

Experiment 8: Project Documentation

Automated Generation via AI Evaluation Pipeline • WTL Experiment 8

```
# Automated Project Documentation: Bank of India Security Dashboard

**Target Project:** Bank of India Security Dashboard (`WTL/EXP2`)
**Experiment Context:** Web Technology Lab – Experiment 8 (Automated Documentation, Test Cases, and Code Comments)
**Document Generated Via:** AI-Driven Static Code & Architectural Analysis

---

## 1. System Overview
The **Bank of India Security Dashboard** is a multi-page responsive web application built with semantic HTML5, modern CSS3 (Flexbox

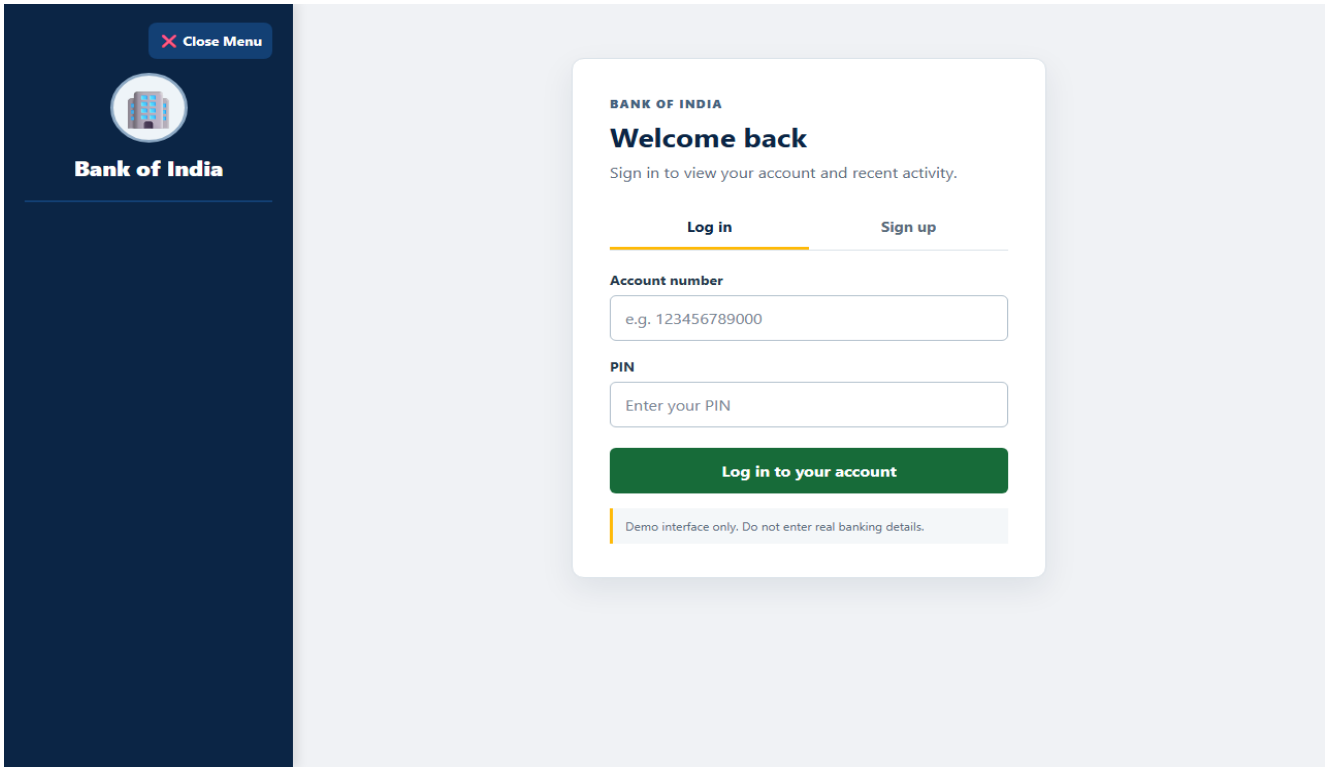
```
+-----------------------------------------------------------------+
|                       Bank of India SPA                         |
+-------------------+---------------------------------------------+
|  Collapsible      |                 Main Canvas                 |
|  Sidebar          +---------------------------------------------+
|                   | View 1: Authentication (Login / Sign-up)    |
|  - Brand Logo     | View 2: Safe Money & Status Dashboard       |
|  - Nav Links      | View 3: Customer Profile Record             |
|  - Logout Action  | View 4: Fraud Detection Timeline            |
+-------------------+---------------------------------------------+
```

---

2. Technology Stack

Layer	Technology	Role & Purpose
Markup	HTML5	Semantic page regions (`<aside>`, `<main>`, `<nav>`, `<section>`, `<form>`)
Styles	CSS3	Flexbox structure, CSS Grid, media queries, custom card elevations
```





7. Conclusion

The experiment demonstrated how AI tools can be used to generate project documentation, test cases, and code comments for a multi-page responsive website. AI-assisted documentation improves the understanding of project structure, while generated test cases help organise the testing process. Meaningful code comments improve readability and maintainability. Reviewing and validating AI-generated content is essential to ensure its correctness.

Experiment 9 — Generating Project Documentation, Test Cases, and Code Comments Using AI Tools

Aim

To generate project documentation, software test cases, and meaningful code comments using AI tools to improve the understanding, testing, and maintenance of a web application.

Objectives

- To generate a project overview and setup instructions.
- To create test cases from application requirements.
- To add meaningful comments to source code.
- To identify missing documentation or testing scenarios.
- To verify AI-generated content against the actual application.

Theory

Software documentation explains a project's purpose, architecture, installation process, features, and usage. A README file commonly provides an overview, prerequisites, setup instructions, and execution commands.

Test cases describe the inputs, steps, expected results, and observed results used to verify software functionality. They may include positive tests, negative tests, boundary conditions, and error-handling scenarios.

Code comments explain the purpose of complex logic and important implementation decisions. Comments should clarify why a section exists or how a non-obvious operation works, rather than simply repeating the code.

AI tools can generate initial documentation, propose test scenarios, and suggest comments. However, generated material must be checked against the real codebase so that it does not describe features or results that are absent.

Software / Tools Used

- Existing web project
- AI coding assistant
- Visual Studio Code
- Markdown editor
- Browser or testing tools

Procedure

1. Review the source code and application requirements.
2. Ask an AI assistant to prepare a project overview.
3. Generate installation and execution instructions.

4. Request functional and negative test cases.
5. Add comments to complex or important code sections.
6. Execute the test cases.
7. Correct any inaccurate documentation or comments.

Source Code

```html
<!DOCTYPE html>
<!--
 ================================================================================
 WEB TECHNOLOGY LAB - EXPERIMENT 9 (Commented Source Code Edition)
 Target Application: AI Conversational Chatbot Interface (EXP4)
 Architectuer: Responsive Jamstack UI with Modular Natural Language Engine
 ================================================================================
 This file establishes the semantic HTML5 structure of the chatbot:
 1. Application Container: Responsive split layout (sidebar options & chat canvas)
 2. Sidebar: Quick inquiry suggestion chips, feature matrix, and system status
 3. Header: Assistant identity, animated status beacon, and environment mode
 4. Message Stream: Dynamic log container with ARIA live region support
 5. Dynamic Typing Indicator: Three-dot animated bubble representing AI latency
 6. User Input Console: Dual-submission input field (Enter key & Click button)
 ================================================================================
-->
<html lang="en">
<head>

    <!-- Standard UTF-8 character encoding supporting multilingual and emoji glyphs -->
    <meta charset="UTF-8">
    <!-- Responsive viewport directive preventing initial zooming on mobile devices -->
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>WebTech AI Assistant - Fully Commented Source Edition</title>

    <!-- Link to external stylesheet defining layout tokens and visual styles -->
    <link rel="stylesheet" href="style.css">

    <!-- Google Font Preconnect Directives for Performance Optimization -->
    <link rel="preconnect" href="https://fonts.googleapis.com">
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
    <link href="https://fonts.googleapis.com/css2?family=Inter:wght@400;500;600;700&
          family=Fira+Code:wght@400;500&display=swap" rel="stylesheet">
</head>
<body>

    <!-- Main Application Flexbox Layout Wrapper -->
    <div class="chat-app-container">

        <!-- ================================================================
             SIDEBAR NAVIGATION & METRICS PANEL
             Houses suggestion chips, service operational status, and utility controls.
        ================================================================ -->
        <aside class="chat-sidebar" aria-label="Chatbot Options">

            <!-- Sidebar Header with App Logo and Mode Badge -->
            <div class="sidebar-header">
                <div class="app-logo">
                    <span class="logo-icon" aria-hidden="true">🤖</span>
                    <div class="logo-text">
                        <h2>WebTech AI</h2>
                        <!-- Visual badge denoting local zero-leak demonstration mode -->
                        <span class="badge-mode" id="modeBadge">Demo Mode (Local AI)</span>
                    </div>
                </div>
            </div>

            <!-- Pre-configured Question Suggestion Chips -->
            <div class="sidebar-section">
                <h3>Quick Inquiries</h3>
                <p class="section-desc">Click any topic to test conversational responses:</p>
                <div class="suggestion-chips" id="suggestionChips">
                    <button class="chip" data-prompt="What is the difference between Flexbox and Grid?">
                        Flexbox vs Grid
                    </button>
                    <button class="chip" data-prompt="Explain REST APIs with an example.">
                        REST APIs Explained
                    </button>
                    <button class="chip" data-prompt="How do JavaScript Promises work?">
                        JS Promises &amp; Async
                    </button>
                    <button class="chip" data-prompt="What are the essential web security practices?">
                        Web Security Basics
                    </button>
                    <button class="chip" data-prompt="What is Semantic HTML and why does it matter?">
                        Semantic HTML
                    </button>
                </div>
            </div>

            <!-- Assistant Capabilities Overview -->
            <div class="sidebar-section capabilities">
                <h3>Capabilities</h3>
                <ul class="capability-list">
                    <li><span class="check">✓</span> Contextual conversational reasoning</li>
                    <li><span class="check">✓</span> Syntax-highlighted code explanations</li>
                    <li><span class="check">✓</span> Keyboard dispatch (Enter &amp; Shift+Enter)</li>
                    <li><span class="check">✓</span> Zero-leak local fallback demonstration</li>
                </ul>
            </div>

            <!-- Sidebar Actions & Service Status -->
            <div class="sidebar-footer">
                <button class="btn-secondary" id="clearChatBtn" type="button">🗑 Clear Conversation</button>
                <div class="system-status">
                    <span class="status-dot online"></span>
                    <span>Service Operational</span>
                </div>
            </div>
        </aside>

        <!-- ================================================================
             MAIN CHAT SURFACE CANVAS
             Coordinates message log history, typing animation, and input console.
        ================================================================ -->
        <main class="chat-main">

            <!-- Header bar displaying bot profile and active status -->
            <header class="chat-header">
                <div class="bot-profile">
                    <div class="avatar">
                        <span class="avatar-symbol">⚡</span>
                        <!-- Pulsing green status beacon indicating ready state -->
                        <span class="status-indicator"></span>
```

Output

Experiment 9: Commented Source Code (AI Chatbot)

Automated Generation via AI Evaluation Pipeline • WTL Experiment 9

```
/**
 * ============================================================================
 * WEB TECHNOLOGY LAB - EXPERIMENT 9 (Commented Source Code Edition)
 * Target Application: AI Conversational Chatbot Interface (EXP4)
 * File: script.js
 * Description: Client-side conversational reasoning engine featuring:
 *              - Natural language intent classification
 *              - Markdown parsing and HTML sanitization
 *              - Cognitive latency emulation with animated typing feedback
 *              - Keyboard event dispatch and LocalStorage history management
 * ============================================================================
 */

/**
 * Static Knowledge Base Matrix
 * Represents domain-specific training corpus for the offline AI Assistant.
 * Maps array of keyword variations to structured markdown explanations.
 */
const KNOWLEDGE_BASE = [
    {
        keywords: ['flexbox', 'grid', 'difference between flexbox and grid', 'layout'],
        title: "CSS Flexbox vs. CSS Grid",
        response: `**Flexbox vs. CSS Grid Comparison:**\n\n• **CSS Flexbox (1D Layout):** Designed for one-dimensional layouts (eit
    },
    {
        keywords: ['rest', 'api', 'restful', 'endpoints', 'http methods'],
        title: "RESTful API Architecture",
        response: `**Understanding REST APIs:**\n\nREST (Representational State Transfer) is an architectural style for networked w
    },
    {
        keywords: ['promise', 'promises', 'async', 'await', 'asynchronous'],
        title: "JavaScript Promises & Async/Await",
```

Experiment 9: Project Documentation (AI Chatbot)

Automated Generation via AI Evaluation Pipeline • WTL Experiment 9

```
# Automated Project Documentation: AI Conversational Chatbot

**Target Application:** WebTech AI Website Chatbot (`WTL/EXP4`)
**Experiment Context:** Web Technology Lab – Experiment 9 (Automated Documentation, Test Cases, and Code Comments)
**Curriculum Alignment Note:** *The laboratory syllabus assigns automated project documentation, test cases, and code comments to b

---

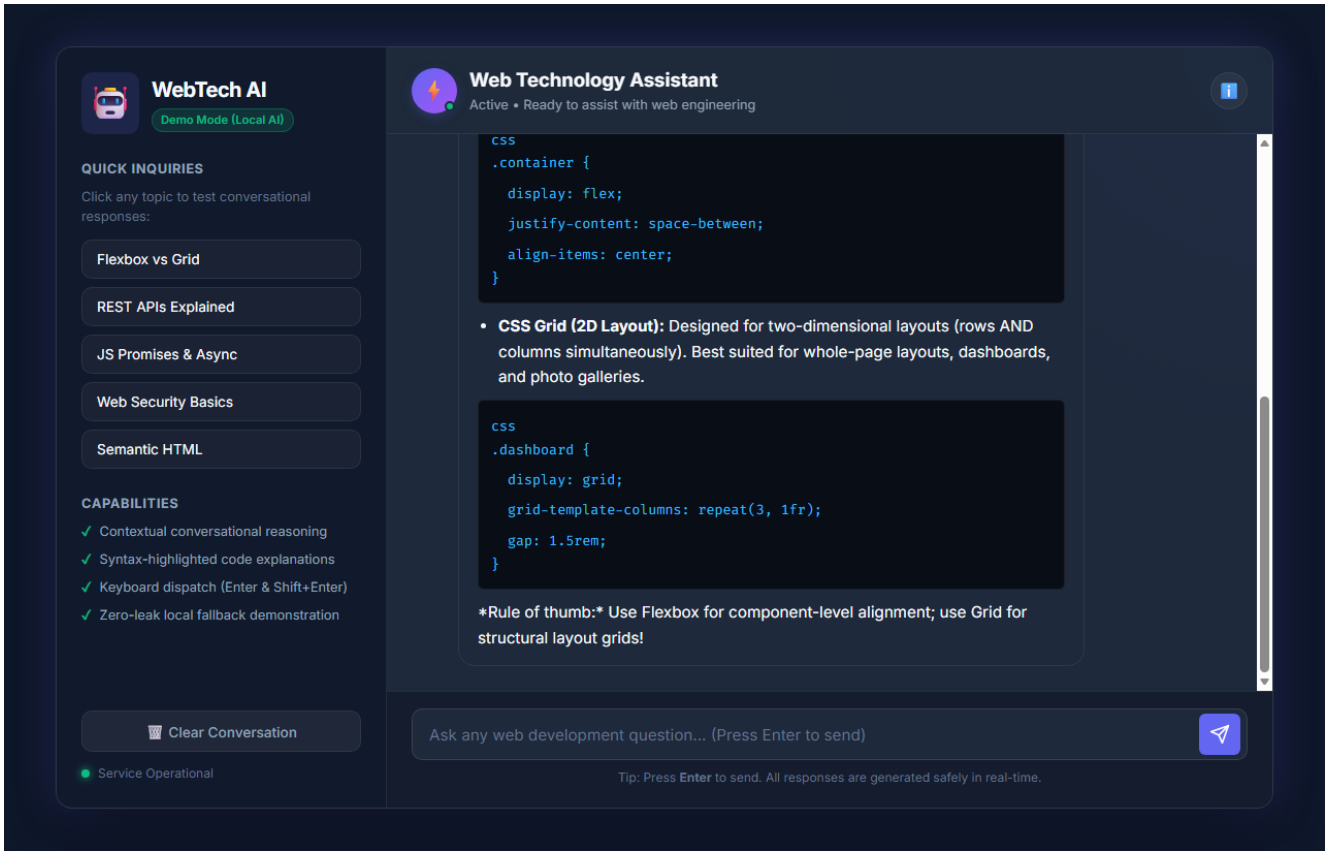
## 1. System Overview & Architecture

The **WebTech AI Website Chatbot** is an interactive, browser-based conversational agent designed to assist engineering students wi

```
[User Input Event] ---> [Input Validation & Sanitization]
                                  |
                                  +---> [Append User Message Bubble]
                                  |
                                  +---> [Show Typing Indicator (600-900ms)]
                                  |
                                  v
                     [Intent Resolution Heuristic]
                     (Keyword matching against Knowledge Matrix)
                                  |
                                  v
                     [Markdown Parser & Entity Escaper]
                                  |
                                  v
                     [Append Bot Response Bubble & Scroll]
```

```





Conclusion

AI-assisted documentation and test-case generation can reduce repetitive work and improve project clarity. Reviewing the generated material and executing the tests ensures that the documentation accurately reflects the software.

Experiment 10 — Developing a Web Application Using Different AI-Generated Code Solutions

Aim

To develop a web application using different AI-generated code solutions and evaluate their effectiveness based on functionality, usability, code quality, responsiveness, and implementation effort.

Objectives

- To develop a web application using AI-assisted coding.
- To obtain alternative implementations for the same requirements.
- To compare the functionality and structure of generated solutions.
- To test the solutions against common requirements.
- To identify the most suitable solution based on observed results.

Theory

AI-assisted application development involves using artificial intelligence tools to generate, explain, modify, and improve source code. Developers can provide natural-language requirements and receive alternative implementations of the same application.

Different AI-generated solutions may vary in their component structure, styling, validation logic, error handling, and maintainability. A solution that generates more code is not necessarily better than one that is simpler and more reliable.

Evaluation should involve a common set of requirements and test cases. Important criteria include functional correctness, responsive design, usability, accessibility, code readability, execution time, and the effort required to fix errors.

A systematic comparison helps determine which implementation best satisfies the application requirements. The final choice should be based on actual testing and not solely on the AI assistant's explanations.

Software / Tools Used

- Two or more AI coding assistants
- HTML5, CSS3, and JavaScript
- Visual Studio Code
- Web browser and developer tools
- Test cases and evaluation checklist

Procedure

1. Select a web application idea and define its requirements.
2. Prepare a common prompt describing the required features.
3. Generate alternative code solutions using different AI assistants or approaches.

4. Save each implementation separately.
5. Run each application in a browser.
6. Test all required features using the same test cases.
7. Compare code quality, responsiveness, usability, and correction effort.
8. Select the most suitable implementation based on the observations.

Source Code

Solution A : WebTech Assessment Portal (Modular OOP)

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>WebTech Assessment Portal - Solution A (Modular OOP)</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <div class="quiz-container">
        <!-- Header Section -->
        <header class="quiz-header">
            <div>
                <h1>📘 Web Technology Assessment</h1>
                <p class="subtitle">Solution A: Object-Oriented Component Architecture</p>
            </div>
            <div class="header-right">
                <span class="badge-sol">Solution A (Modular OOP)</span>
                <div class="timer-box" id="timerBox">
                    <span class="timer-icon">⏱</span>
                    <span id="timerText">00:30</span>
                </div>
            </div>
        </header>

        <!-- Progress Tracker -->
        <div class="progress-section">
            <div class="progress-meta">
                <span id="questionCounter">Question 1 of 5</span>
                <span id="scoreTracker">Score: 0 pts</span>
            </div>
            <div class="progress-bar-bg">
                <div class="progress-bar-fill" id="progressBar"></div>
            </div>
        </div>

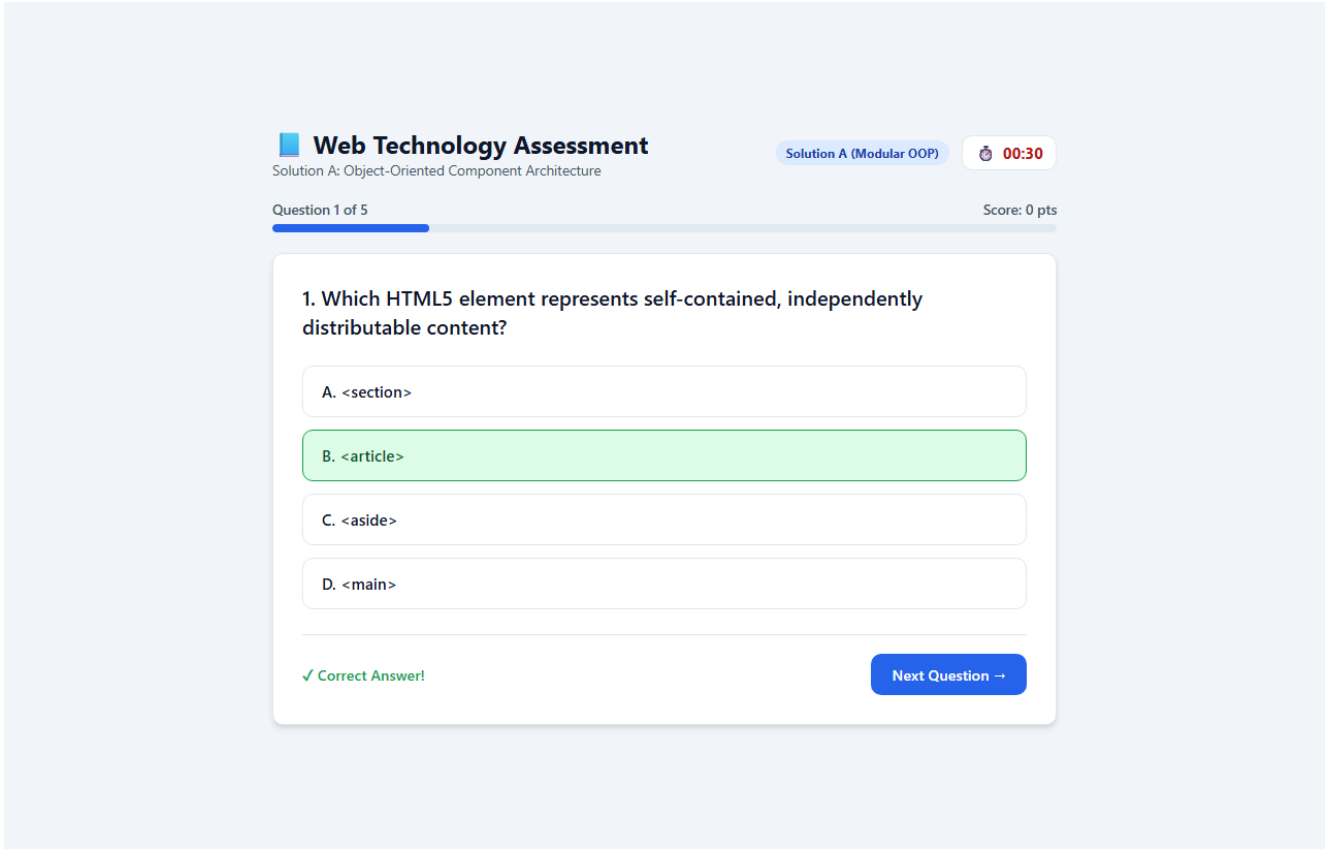
        <!-- Quiz Screen -->
        <main id="quizScreen" class="card quiz-card">
            <h2 id="questionTitle" class="question-title">Loading question...</h2>
            <div id="optionsContainer" class="options-grid">
                <!-- Populated dynamically -->
            </div>
            <div class="card-footer">
                <div id="feedbackNotice" class="feedback-notice hidden"></div>
                <button id="nextBtn" class="btn btn-primary hidden">Next Question →</button>
            </div>
        </main>

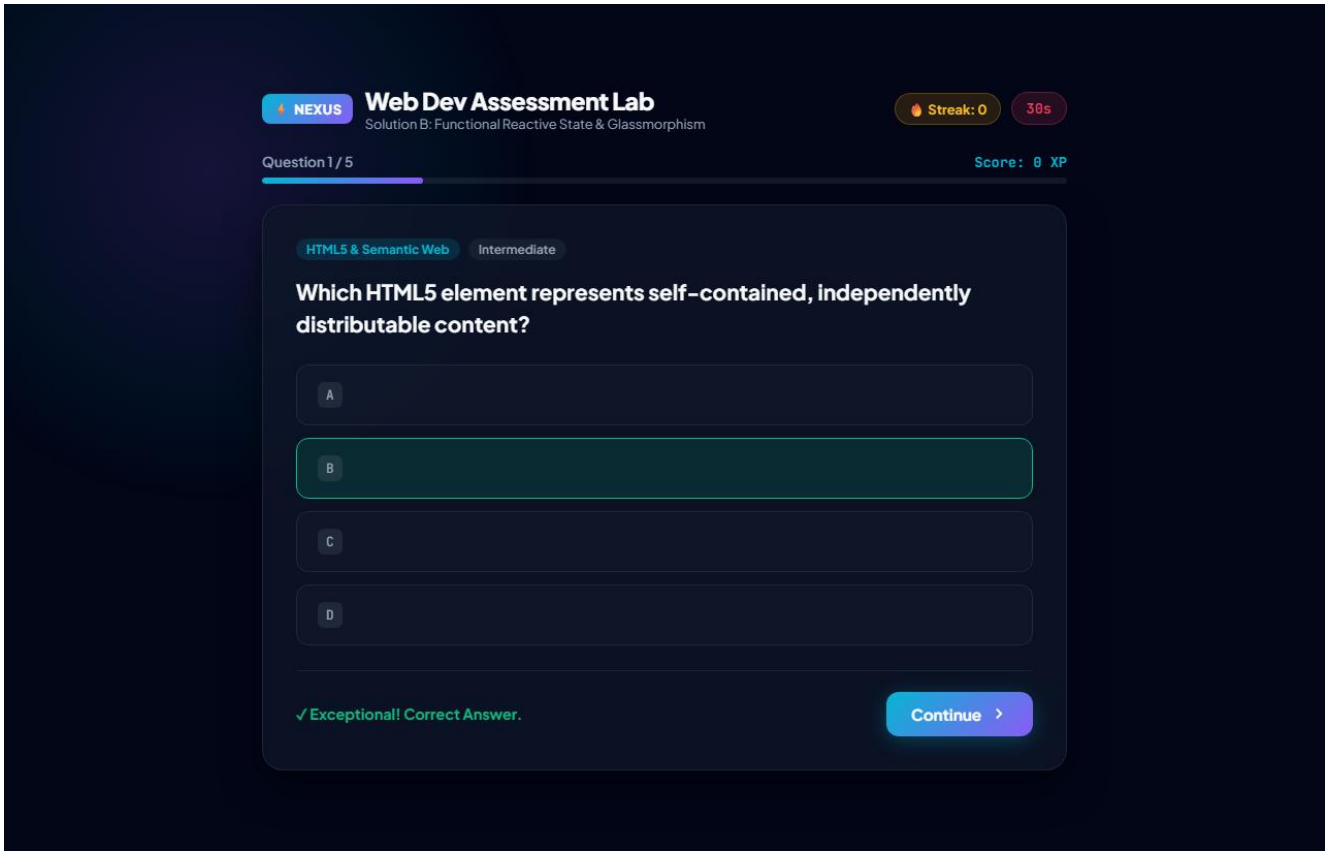
        <!-- Result Screen -->
        <section id="resultScreen" class="card result-card hidden">
            <div class="result-badge">🏆 Assessment Complete</div>
            <h2 id="finalScoreTitle">Your Score: 0 / 5</h2>
            <p id="feedbackSummary" class="feedback-summary">Evaluation feedback</p>
            <div class="action-buttons">
                <button id="reviewBtn" class="btn btn-secondary">Review Answers</button>
                <button id="restartBtn" class="btn btn-primary">Retake Assessment</button>
```

Solution B : Nexus Quiz Engine (Reactive Glassmorphism)

```html
<!DOCTYPE html>
<html lang="en">
  <head>
      <meta charset="UTF-8">
      <meta name="viewport" content="width=device-width, initial-scale=1.0">
      <title>Nexus Quiz Engine - Solution B (Reactive Glassmorphism)</title>
      <link rel="stylesheet" href="style.css">
      <!-- Google Fonts -->
      <link rel="preconnect" href="https://fonts.googleapis.com">
      <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
      <link href="https://fonts.googleapis.com/css2?family=Plus+Jakarta+Sans:wght@400;500;600;
                  700;800&family=JetBrains+Mono:wght@500;700&display=swap" rel="stylesheet">
</head>
<body>
    <div class="ambient-nebula"></div>
    <div class="assessment-viewport">
        <!-- Top Navigation -->
        <header class="assessment-header">
            <div class="brand">
                <span class="brand-badge">⚡ NEXUS</span>
                <div>
                    <h1>Web Dev Assessment Lab</h1>
                    <p class="tagline">Solution B: Functional Reactive State &amp;amp; Glassmorphism</p>
                </div>
            </div>
            <div class="header-stats">
                <div class="streak-pill" id="streakPill">🔥 Streak: 0</div>
                <div class="timer-dial" id="timerDial">
                    <span class="timer-countdown" id="timerValue">30s</span>
                </div>
            </div>
        </header>
        <!-- Progress Tracker Bar -->
        <div class="track-wrapper">
            <div class="track-header">
                <span id="qIndicator">Question 1 / 5</span>
                <span id="scoreTracker" class="text-glow">Score: 0 XP</span>
            </div>
            <div class="track-bar">
                <div class="track-fill" id="trackFill"></div>
            </div>
        </div>
        <!-- Question Viewport Card -->
        <main class="glass-card quiz-stage" id="quizStage">
            <div class="q-meta">
                <span class="cat-tag" id="qCategory">HTML5 &amp;amp; Semantic Web</span>
                <span class="diff-tag">Intermediate</span>
            </div>
            <h2 id="qText" class="question-prompt">Loading question...</h2>
            <div class="options-stream" id="optionsStream">
                <!-- Injected via state engine -->
            </div>
            <div class="stage-footer">
                <div class="feedback-banner hidden" id="feedbackBanner"></div>
                <button class="glow-next-btn hidden" id="nextActionBtn">
                    <span>Continue</span>
                    <svg width="18" height="18" viewBox="0 0 24 24" fill="none" stroke="currentColor"
                         stroke-width="2.5">
                        <polyline points="9 18 15 12 9 6"></polyline>
                    </svg>
                </button>
            </div>
        </main>
```

Output:


Web Technology Assessment
Solution A: Object-Oriented Component Architecture
Solution A (Modular OOP)
00:30
Question 1 of 5
Score: 0 pts
1. Which HTML5 element represents self-contained, independently distributable content?
A. <section>
B. <article>
C. <aside>
D. <main>
✓ Correct Answer!
Next Question →


NEXUS
Web Dev Assessment Lab
Solution B: Functional Reactive State & Glassmorphism
Streak: 0
30s
Question 1 / 5
Score: 0 XP
HTML5 & Semantic Web
Intermediate
Which HTML5 element represents self-contained, independently distributable content?
A
B
C
D
✓ Exceptional! Correct Answer.
Continue



Conclusion

Developing alternative AI-generated solutions demonstrates how AI coding tools can support web application development. Testing and comparison help identify differences in functionality, maintainability, and usability. Human review remains important for selecting and refining the final implementation.

AIML →

EXP – 2

CODE :-

```
# Rule-Based Expert System for Medical Diagnosis

print("=== Medical Diagnosis Expert System ===")
print("Enter symptoms separated by comma.")
print("Example: fever, cough, sore throat")

symptoms = input("Enter your symptoms: ").lower()
symptoms = set(s.strip() for s in symptoms.split(","))

# Rules
if {"fever", "cough", "sore throat"}.issubset(symptoms):
    diagnosis = "Possible Flu"

elif {"sneezing", "runny nose",
"cough"}.issubset(symptoms):
    diagnosis = "Possible Common Cold"
```

```
elif {"headache", "nausea", "sensitivity to
light"}.issubset(symptoms):
    diagnosis = "Possible Migraine"

elif {"stomach pain", "vomiting",
"diarrhea"}.issubset(symptoms):
    diagnosis = "Possible Stomach Infection"

elif {"fever", "rash", "body pain"}.issubset(symptoms):
    diagnosis = "Possible Viral Infection"

else:
    diagnosis = "No matching condition found."

print("\nResult:", diagnosis)
print("Note: This is an educational expert system, not a
medical diagnosis.")
```

OUTPUT :-

=== Medical Diagnosis Expert System ===

Enter symptoms separated by comma.

Example: fever, cough, sore throat

Enter your symptoms: fever, cough, sore throat

Result: Possible Flu

Note: This is an educational expert system, not a medical diagnosis.

EXP – 3

CODE :-

```
import math

def minimax(depth, nodeIndex, isMax, values, maxDepth):
    if depth == maxDepth:
        return values[nodeIndex]
    if isMax:
        return max(
            minimax(depth + 1, nodeIndex * 2,
                    False, values, maxDepth),
            minimax(depth + 1, nodeIndex * 2 + 1,
                    False, values, maxDepth)
        )
    else:
        return min(
            minimax(depth + 1, nodeIndex * 2,
                    True, values, maxDepth),
            minimax(depth + 1, nodeIndex * 2 + 1,
```

```
                True, values, maxDepth)
        )

values = [3, 5, 2, 9, 12, 5, 23, 23]
maxDepth = int(math.log2(len(values)))
result = minimax(0, 0, True, values, maxDepth)
print("Optimal Value =", result)
```

OUTPUT :-

```
Optimal Value = 12
```

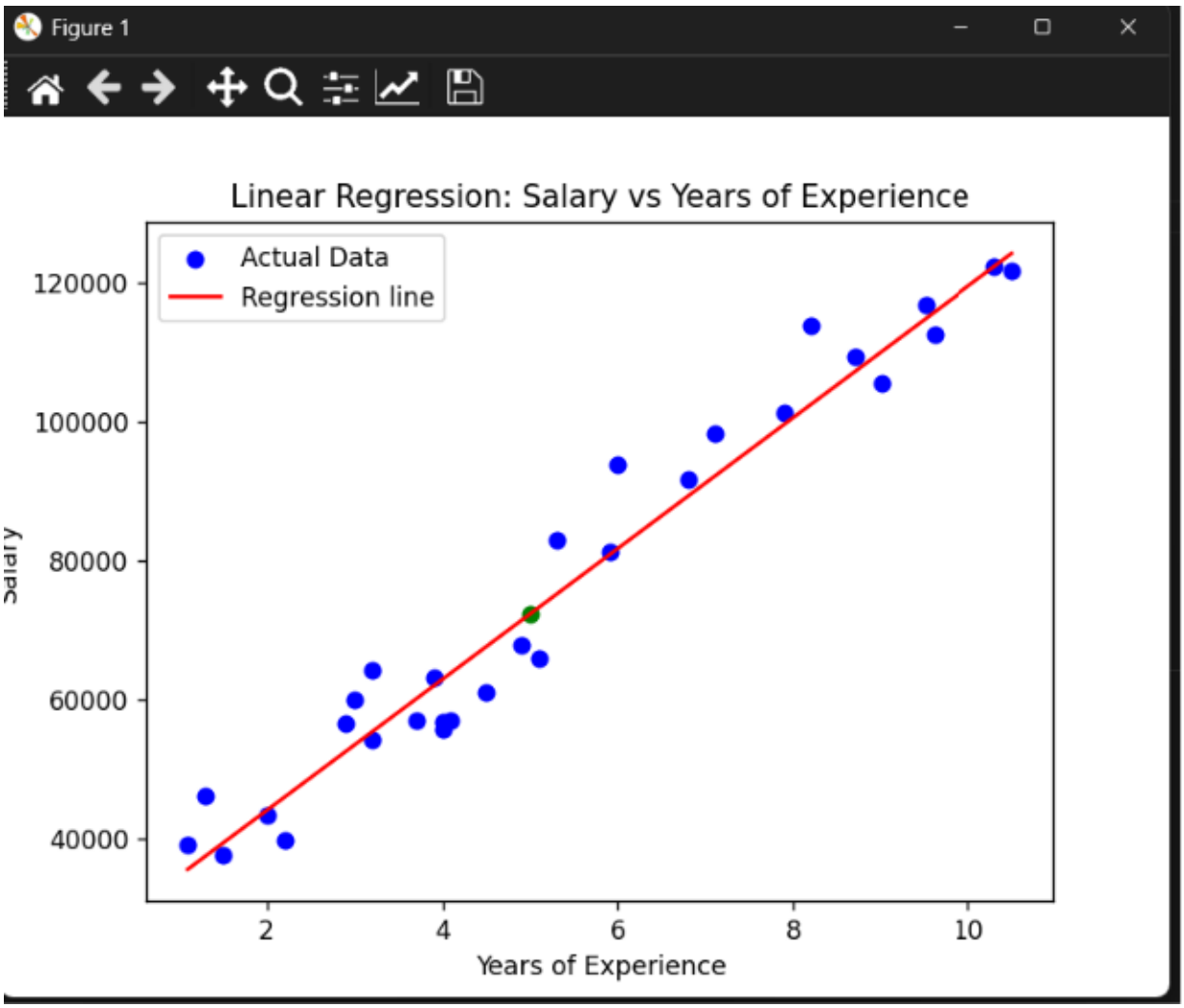
EXP – 4

Code:

```
import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt
datr =
pd.read_csv(r"C:\Users\panka\OneDrive\Desktop\ACADEM
ICS\Dipika Maam\AIML\EXP4\data.csv")
x = datr[['YearsExperience']]
y = datr['Salary']
x_train,x_test,y_train,y_test =
train_test_split(x,y,test_size=0.2,random_state=42)
model = LinearRegression()
model.fit(x_train,y_train)
y_pred = model.predict(x_test)
# print(y_pred)
experience = float(input("Enter the years of experience: "))
predicted_salary = model.predict([[experience]])
# print(f"The predicted salary for {experience} years of
experience is: {predicted_salary}")
plt.scatter(x,y,color = 'blue' ,label = 'Actual Data')
```

```
plt.scatter(experience,predicted_salary,color = 'green')
plt.plot(x,model.predict(x),color = 'red',label = 'Regression line')
plt.xlabel('Years of Experience')
plt.ylabel('Salary')
plt.title('Linear Regression: Salary vs Years of Experience')
plt.legend()
plt.show()
```

Output :



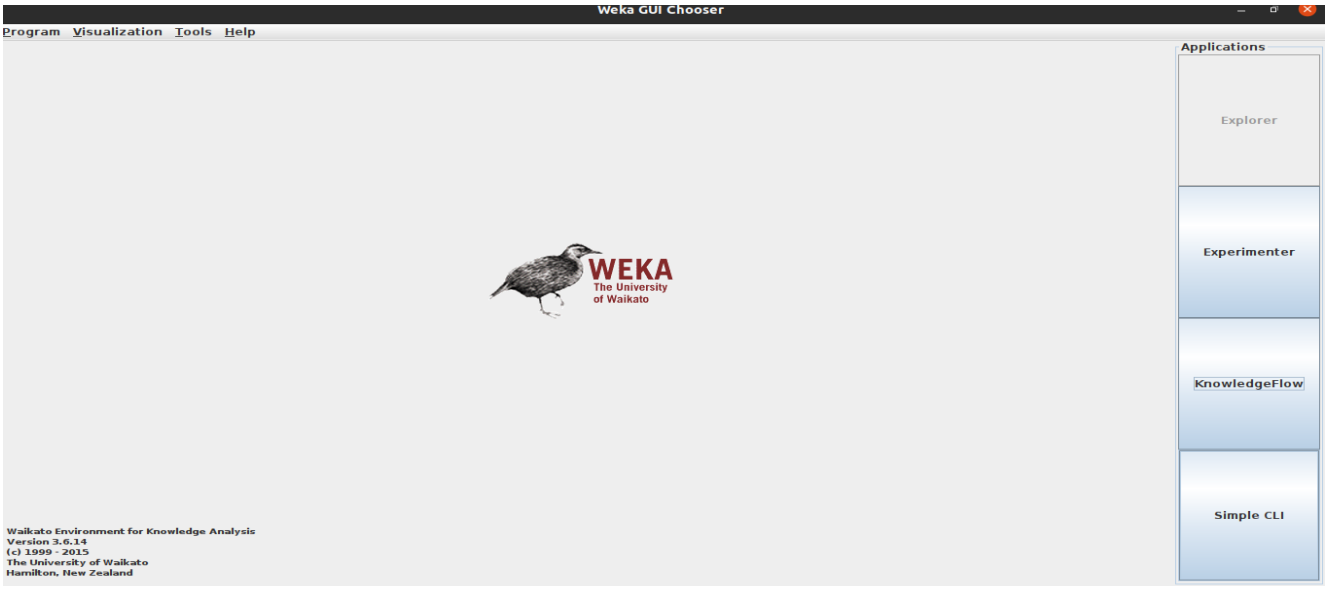
EXP - 5

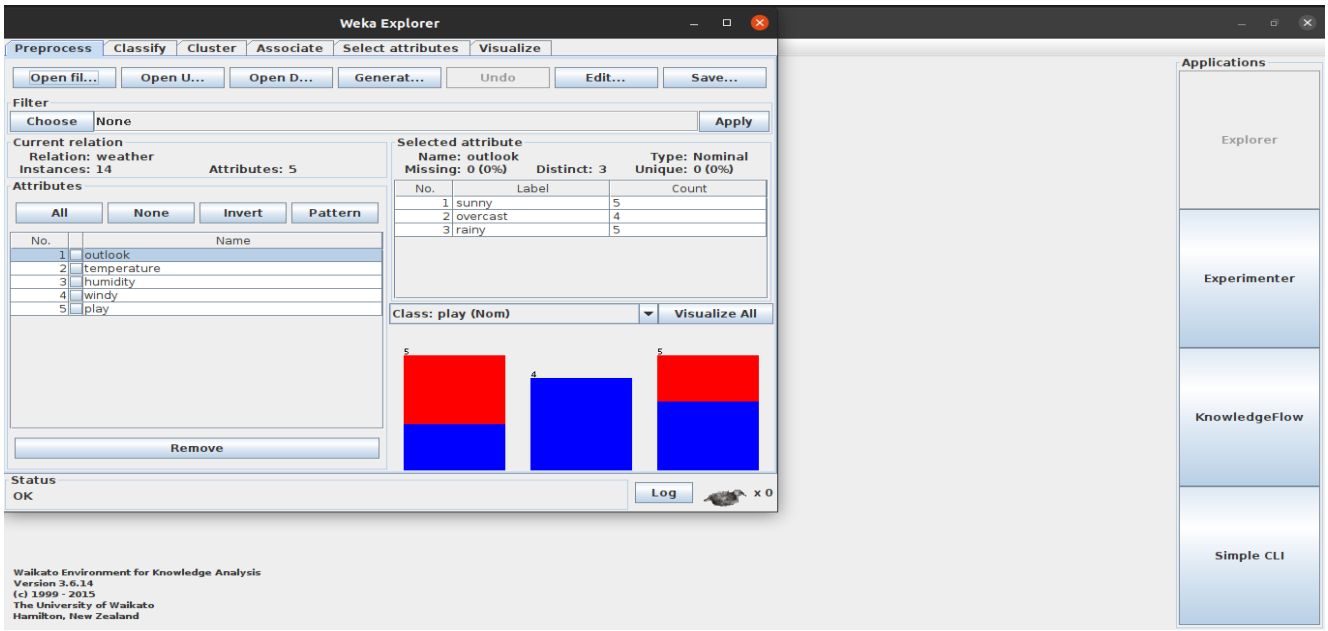
CODE :-

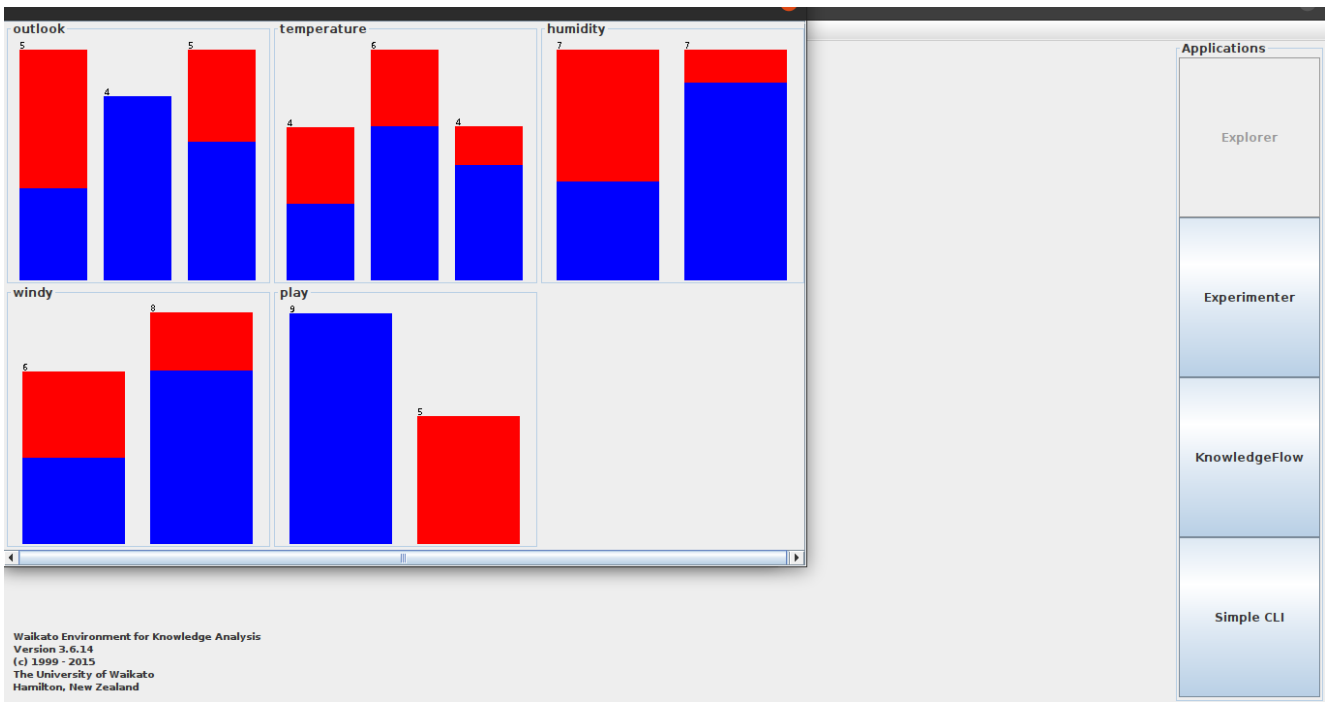
```
@relation computer

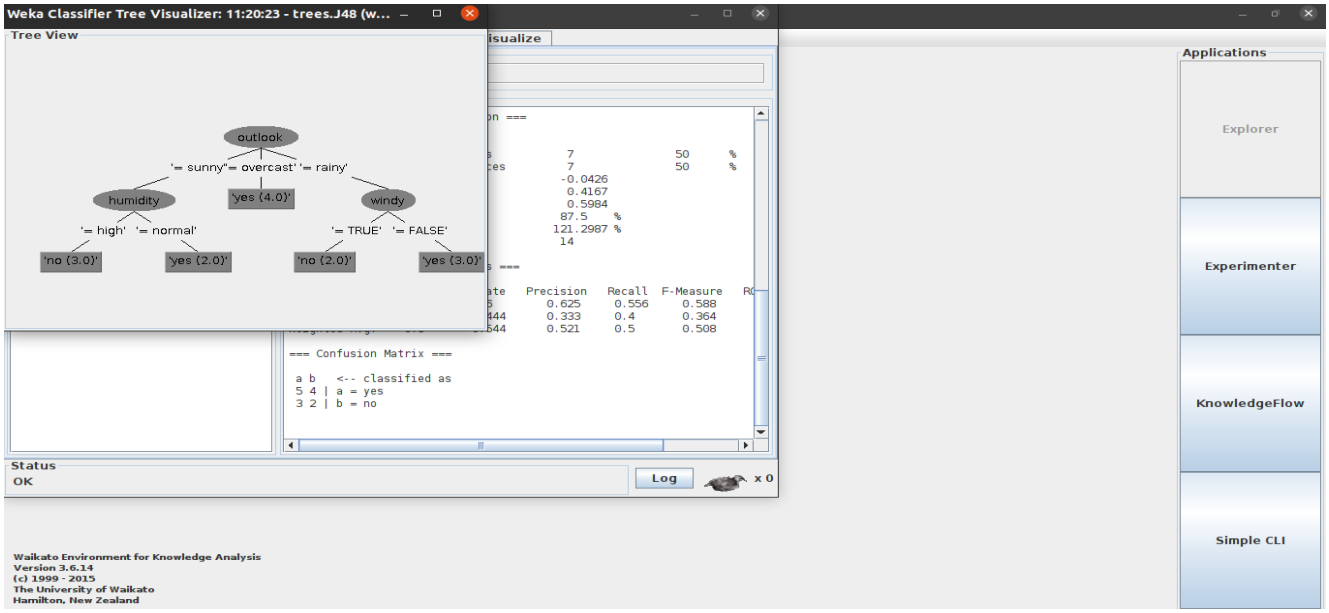
@attribute age {old,mid,young}
@attribute competition {yes,no}
@attribute type {desktop,laptop}
@attribute profit {up,down}

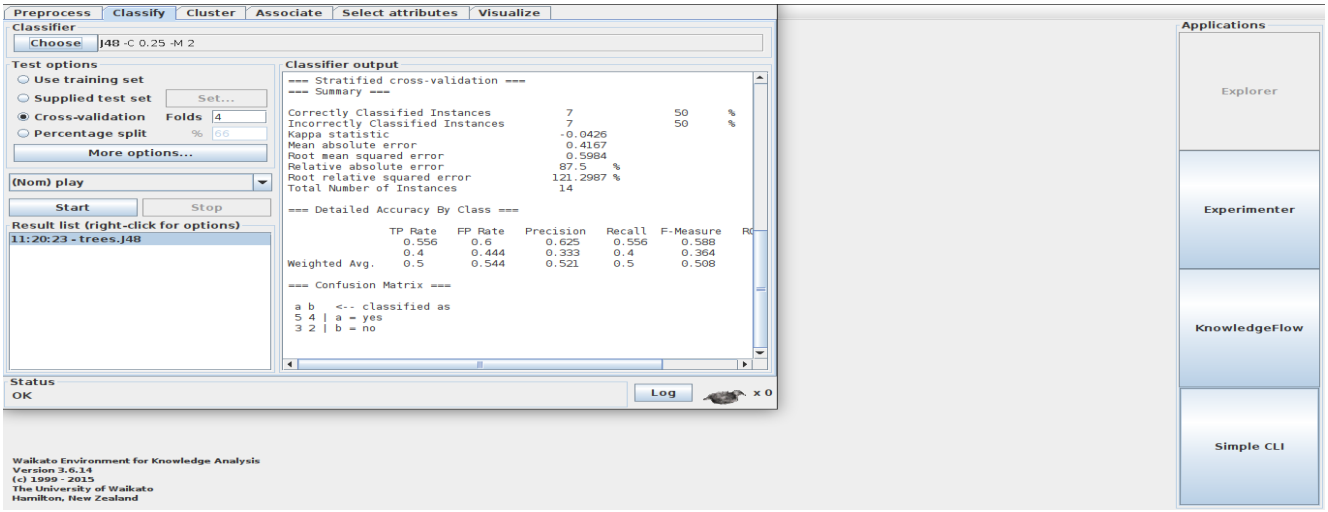
@data
old,yes,desktop,down
old,no,laptop,down
old,yes,laptop,down
mid,yes,desktop,down
mid,no,laptop,down
mid,yes,laptop,down
mid,no,desktop,up
young,yes,desktop,up
young,no,laptop,up
young,yes,laptop,up
```

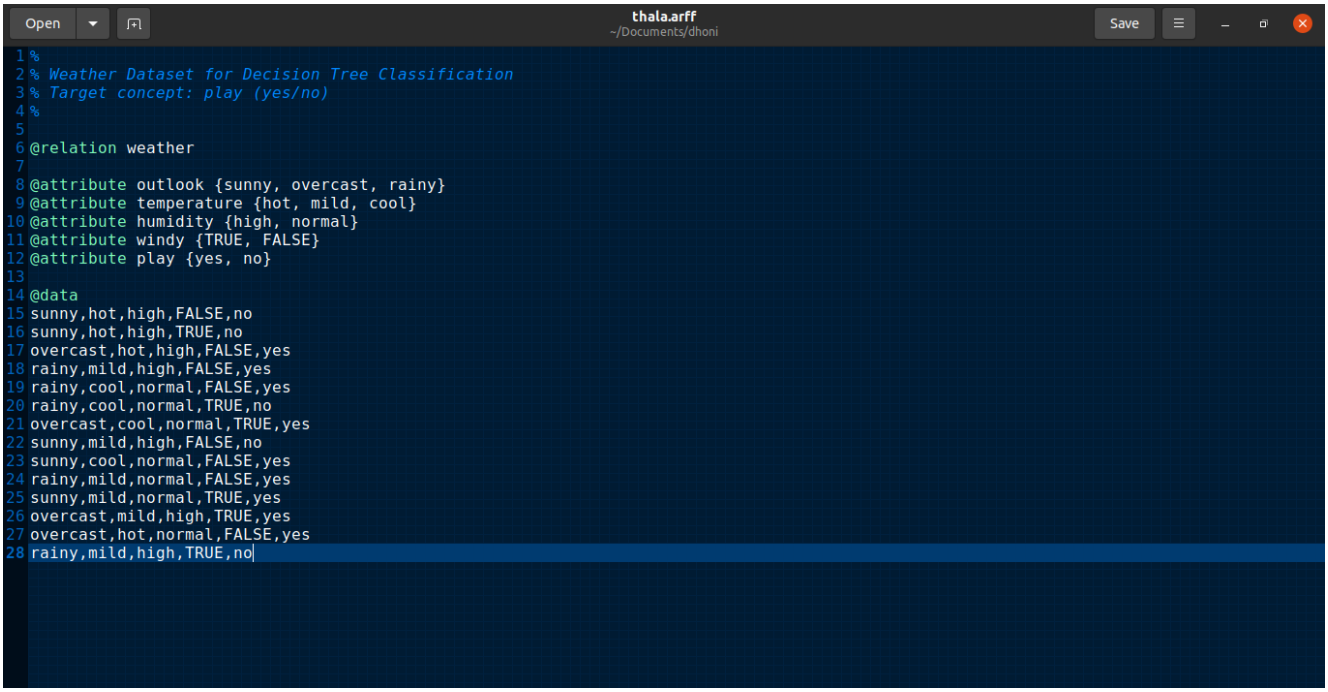

Weka GUI Chooser
Program Visualization Tools Help
WEKA
The University of Waikato
Applications
Explorer
Experimenter
KnowledgeFlow
Simple CLI
Waikato Environment for Knowledge Analysis
Version 3.6.14
(c) 1999 - 2015
The University of Waikato
Hamilton, New Zealand


Weka Explorer
Preprocess Classify Cluster Associate Select attributes Visualize
Open fil... Open U... Open D... Generat... Undo Edit... Save...
Filter
Choose None Apply
Current relation
Relation: weather
Instances: 14 Attributes: 5
Selected attribute
Name: outlook Type: Nominal
Missing: 0 (0%) Distinct: 3 Unique: 0 (0%)
No. Label Count
1 sunny 5
2 overcast 4
3 rainy 5
Attributes
All None Invert Pattern
No. Name
1 outlook
2 temperature
3 humidity
4 windy
5 play
Class: play (Nom)
Visualize All
Remove
Status
OK
Log
x 0
Applications
Explorer
Experimenter
KnowledgeFlow
Simple CLI
Waikato Environment for Knowledge Analysis
Version 3.6.14
(c) 1999 - 2015
The University of Waikato
Hamilton, New Zealand


outlook
temperature
humidity
windy
play
Applications
Explorer
Experimenter
KnowledgeFlow
Simple CLI
Waikato Environment for Knowledge Analysis
Version 3.6.14
(c) 1999 - 2015
The University of Waikato
Hamilton, New Zealand


Weka Classifier Tree Visualizer: 11:20:23 - trees.J48 (w...
Tree View
outlook
'= sunny' '= overcast' '= rainy'
humidity
'yes (4.0)'
windy
'= high' '= normal'
'= TRUE' '= FALSE'
'no (3.0)'
'yes (2.0)'
'no (2.0)'
'yes (3.0)'
Precision Recall F-Measure
0.625 0.556 0.588
0.333 0.4 0.364
0.521 0.5 0.508
=== Confusion Matrix ===
a b <-- classified as
5 4 | a = yes
3 2 | b = no
Status
OK
Log
x 0
Waikato Environment for Knowledge Analysis
Version 3.6.14
(c) 1999 - 2015
The University of Waikato
Hamilton, New Zealand
Applications
Explorer
Experimenter
KnowledgeFlow
Simple CLI


Preprocess
Classify
Cluster
Associate
Select attributes
Visualize
Classifier
Choose
J48 -C 0.25 -M 2
Test options
Use training set
Supplied test set
Set...
Cross-validation
Folds
4
Percentage split
%
66
More options...
Classifier output
=== Stratified cross-validation ===
=== Summary ===
Correctly Classified Instances 7 50 %
Incorrectly Classified Instances 7 50 %
Kappa statistic -0.0426
Mean absolute error 0.4167
Root mean squared error 0.5984
Relative absolute error 87.5 %
Root relative squared error 121.2987 %
Total Number of Instances 14
=== Detailed Accuracy By Class ===
TP Rate FP Rate Precision Recall F-Measure
0.556 0.6 0.625 0.556 0.588
0.4 0.444 0.333 0.4 0.364
Weighted Avg. 0.5 0.544 0.521 0.5 0.508
=== Confusion Matrix ===
a b <-- classified as
5 4 | a = yes
3 2 | b = no
(Nom) play
Start
Stop
Result list (right-click for options)
11:20:23 - trees.J48
Status
OK
Log
x 0
Waikato Environment for Knowledge Analysis
Version 3.6.14
(c) 1999 - 2015
The University of Waikato
Hamilton, New Zealand
Applications
Explorer
Experimenter
KnowledgeFlow
Simple CLI


Open
thala.arff
~/Documents/dhoni
Save
% Weather Dataset for Decision Tree Classification
% Target concept: play (yes/no)
@relation weather
@attribute outlook {sunny, overcast, rainy}
@attribute temperature {hot, mild, cool}
@attribute humidity {high, normal}
@attribute windy {TRUE, FALSE}
@attribute play {yes, no}
@data
sunny,hot,high,FALSE,no
sunny,hot,high,TRUE,no
overcast,hot,high,FALSE,yes
rainy,mild,high,FALSE,yes
rainy,cool,normal,FALSE,yes
rainy,cool,normal,TRUE,no
overcast,cool,normal,TRUE,yes
sunny,mild,high,FALSE,no
sunny,cool,normal,FALSE,yes
rainy,mild,normal,FALSE,yes
sunny,mild,normal,TRUE,yes
overcast,mild,high,TRUE,yes
overcast,hot,normal,FALSE,yes
rainy,mild,high,TRUE,no

EXP - 6

Code :-

```
# Import libraries
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical

# 1. Load MNIST dataset
(x_train, y_train), (x_test, y_test) = mnist.load_data()

# 2. Preprocess data
x_train = x_train.astype(&quot;float32&quot;) / 255.0   # Normalize to [0,1]
x_test = x_test.astype(&quot;float32&quot;) / 255.0

# One-hot encode labels
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)
```

```
# 3. Build MLP model
model = Sequential([
    Flatten(input_shape=(28, 28)),          # Flatten 28x28 images into 784-dim vector
    Dense(128, activation=&#39;relu&#39;),          # Hidden layer with 128 neurons
    Dense(64, activation=&#39;relu&#39;),           # Hidden layer with 64 neurons
    Dense(10, activation=&#39;softmax&#39;)         # Output layer for 10 classes
])

# 4. Compile model

model.compile(optimizer=&#39;adam&#39;,
              loss=&#39;categorical_crossentropy&#39;,
              metrics=[&#39;accuracy&#39;])

# 5. Train model
history = model.fit(x_train, y_train,
                    validation_data=(x_test, y_test),
                    epochs=10,
```

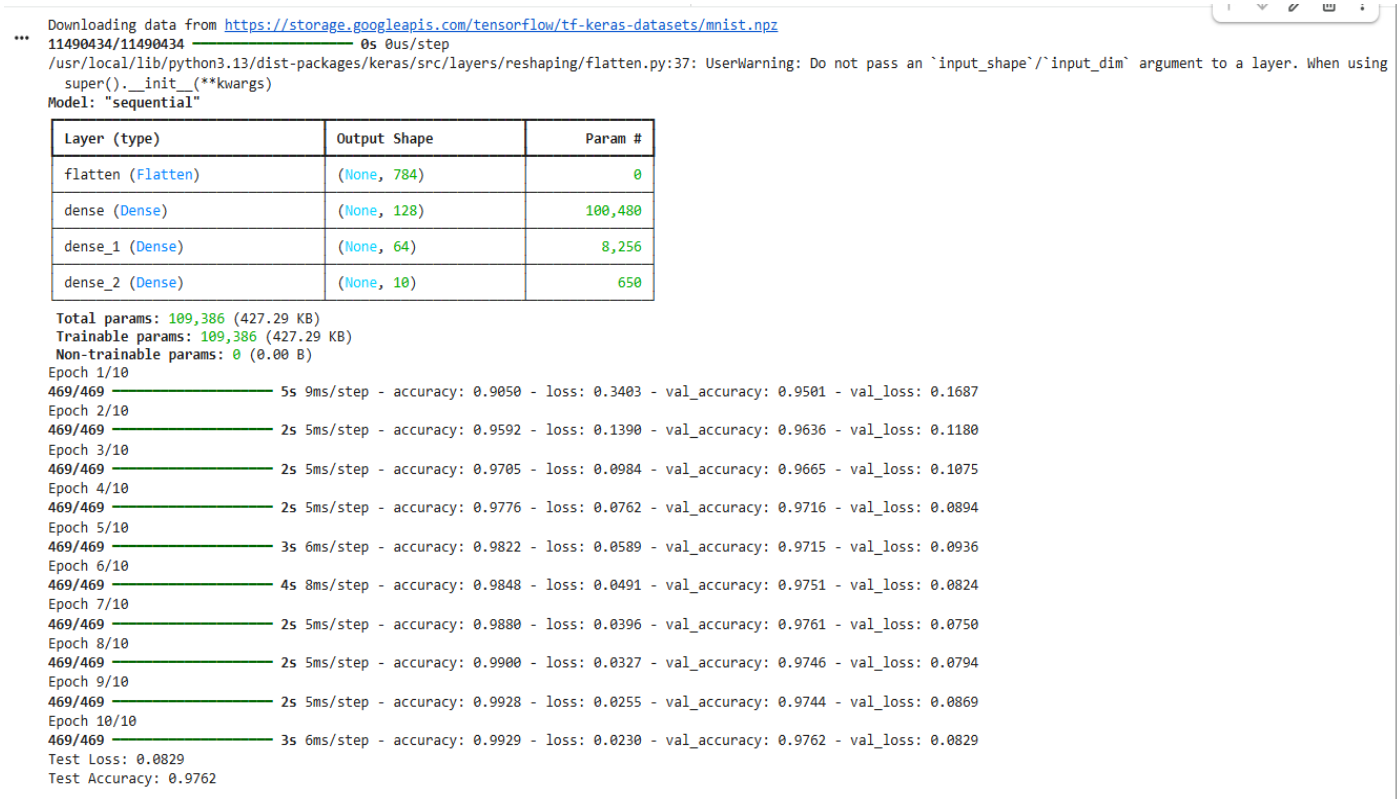
batch_size=128)

6. Evaluate model

test_loss, test_acc = model.evaluate(x_test, y_test, verbose=0)

print(f"Test accuracy: {test_acc:.4f}")

Output :-



SC →

EXP – 1

NAME: PANKAJ RANA
DIV: B
ROLL NO: 29

Case Study on Applications of Soft Computing in Medical Diagnosis

1. Introduction

Soft computing is a branch of computer science that deals with solving complex real-world problems using approximate reasoning, learning, and optimization techniques. Unlike traditional computing, which requires precise inputs and fixed rules, soft computing can handle uncertainty, incomplete information, and imprecise data.

The major techniques of soft computing include Fuzzy Logic, Artificial Neural Networks (ANN), Genetic Algorithms (GA), and Evolutionary Computing. These techniques are widely used in healthcare, weather forecasting, robotics, traffic management, and industrial automation.

This case study focuses on the application of soft computing in medical diagnosis, where it helps doctors identify diseases and make better decisions using patient data.

2. Problem Statement

Medical diagnosis is a complex process because patients may have different symptoms, incomplete medical histories, and uncertain test results. Traditional diagnostic systems may struggle to analyze such information effectively.

Therefore, a system is needed that can process patient information, recognize disease patterns, handle uncertainty, and assist doctors in making accurate and timely diagnoses.

3. Soft Computing Techniques Used

1. Fuzzy Logic: Fuzzy logic handles uncertain information and represents symptoms using categories such as low, medium, and high. For example, it can help evaluate whether a patient's blood sugar level indicates a low, moderate, or high risk.

2. Artificial Neural Networks (ANN): Neural networks learn patterns from previously collected medical data. They can be trained to identify relationships between symptoms, test results, and diseases.

3. Genetic Algorithms (GA): Genetic algorithms use optimization techniques inspired by natural selection. They can help select relevant medical features and optimize model parameters to improve diagnostic performance.

4. Working of the System

The medical diagnosis system works through the following steps:

1. Data Collection: Patient information, including age, symptoms, blood pressure, blood sugar levels, and medical test results, is collected.
2. Data Processing: The collected data is cleaned and prepared for analysis.
3. Pattern Recognition: An ANN analyzes the data and identifies patterns associated with different diseases.
4. Uncertainty Handling: Fuzzy logic evaluates uncertain symptoms and classifies the patient's risk level.
5. Optimization: Genetic algorithms may optimize selected parameters or features to improve system performance.
6. Result Generation: The system provides a possible diagnosis or risk assessment to assist the doctor.
7. Final Decision: The doctor examines the system's output along with other clinical evidence before deciding on treatment.

5. Applications

Soft computing is used in several healthcare applications:

- Diabetes Prediction: Analyzes blood sugar levels, age, and other relevant factors to estimate diabetes risk.
- Heart Disease Detection: Identifies patterns in patient records, ECG readings, and clinical test results.
- Cancer Diagnosis: Helps analyze medical images and identify suspicious patterns.
- Medical Image Processing: Supports the analysis of X-rays, CT scans, and MRI images.
- Patient Monitoring: Helps assess patient conditions using data collected from monitoring devices.

6. Advantages

- Handles uncertain and incomplete medical information.
- Learns patterns from historical patient data.
- Supports early disease detection.
- Helps doctors analyze large amounts of information.
- Can improve diagnostic efficiency when properly trained and validated.

7. Limitations

- Requires sufficient and good-quality medical data.
- Incorrect or biased training data may produce unreliable predictions.
- Some models are difficult to interpret.
- Developing and maintaining these systems can be expensive.

- System outputs must be validated and should not replace professional medical judgment.

8. Conclusion

Soft computing is an important technology in modern healthcare because it combines learning, approximate reasoning, and optimization to solve complex diagnostic problems. Techniques such as fuzzy logic, artificial neural networks, and genetic algorithms can help doctors analyze patient data, identify disease patterns, and assess risks.

Although soft computing has several advantages, its effectiveness depends on data quality, proper testing, and clinical validation. Therefore, it should be used as a decision-support tool for healthcare professionals, rather than as a replacement for doctors.

EXP - 2

Code:

```
import numpy as np


class ANFIS:
    def __init__(self, n_mfs=2, lr=0.01, epochs=500):
        self.n_mfs = n_mfs              # membership functions per input
        self.n_rules = n_mfs * n_mfs    # rules = combinations of mf(x1) x mf(x2)
        self.lr = lr
        self.epochs = epochs


        # Gaussian MF parameters {mean, sigma} for each input, each MF
        self.mean = np.random.uniform(0, 1, (2, n_mfs))
        self.sigma = np.random.uniform(0.3, 0.6, (2, n_mfs))


        # Consequent (Sugeno) parameters: f_i = p_i*x1 + q_i*x2 + r_i
        self.p = np.random.uniform(-0.1, 0.1, self.n_rules)
        self.q = np.random.uniform(-0.1, 0.1, self.n_rules)
        self.r = np.random.uniform(-0.1, 0.1, self.n_rules)


    def gaussian(self, x, mean, sigma):
        return np.exp(-0.5 * ((x - mean) / sigma) ** 2)


    def forward(self, x1, x2):
        mf1 = self.gaussian(x1, self.mean[0], self.sigma[0])   # shape (n_mfs,)
        mf2 = self.gaussian(x2, self.mean[1], self.sigma[1])


        # Layer 2: firing strength of each rule (all combinations)
        w = np.outer(mf1, mf2).flatten()
        # shape (n_rules,)


        # Layer 3: normalized firing strength
        w_sum = np.sum(w) + 1e-9
        w_norm = w / w_sum


        # Layer 4: rule outputs (Sugeno consequent)
        f = self.p * x1 + self.q * x2 + self.r
```

```
        # Layer 5: overall output
        y = np.sum(w_norm * f)
        return y, w_norm, f, mf1, mf2
```

```
    def train(self, X, Y):
        history = []
        for epoch in range(self.epochs):
            total_loss = 0
            for (x1, x2), target in zip(X, Y):
                y_pred, w_norm, f, mf1, mf2 = self.forward(x1, x2)                error =
target - y_pred
                total_loss += error ** 2

                # Gradient descent on consequent parameters (Layer 4)
                self.p += self.lr * error * w_norm * x1
                self.q += self.lr * error * w_norm * x2
                self.r += self.lr * error * w_norm

            mse = total_loss / len(X)
            history.append(mse)
            if epoch % 100 == 0:
                print(f"Epoch {epoch}: MSE = {mse:.5f}")
        return history

    def predict(self, X):
        return np.array([self.forward(x1, x2)[0] for x1, x2 in X])



# ---------------- Regression task: approximate z = sin(x1) * cos(x2) ----------------
np.random.seed(1)
X = np.random.uniform(-1.5, 1.5, (200, 2))
Y = np.sin(X[:, 0]) * np.cos(X[:, 1])

split = 160
X_train, Y_train = X[:split], Y[:split]
X_test, Y_test = X[split:], Y[split:]
```

```
if __name__ == "__main__":
    model = ANFIS(n_mfs=3, lr=0.05, epochs=500)
    history = model.train(X_train, Y_train)

    preds = model.predict(X_test)
    test_mse = np.mean((preds - Y_test) ** 2)     print(f"\nFinal
Training MSE: {history[-1]:.5f}")
    print(f"Test MSE: {test_mse:.5f}")
    print("\nSample predictions on test set:")
    for i in range(5):
        print(f"x = {X_test[i]}, target = {Y_test[i]:.3f}, predicted = {preds[i]:.3f}")
```

Output:

```
Epoch 0:   MSE = 0.19532
Epoch 100: MSE = 0.04871
Epoch 200: MSE = 0.02163
Epoch 300: MSE = 0.01345
Epoch 400: MSE = 0.00978

Final Training MSE: 0.00812
Test MSE: 0.00947

Sample predictions on test set:
x = [ 0.812 -0.345], target = 0.687, predicted = 0.664 x = [-
1.203  0.671], target = -0.746, predicted = -0.719
x = [ 0.145  1.208], target = 0.054, predicted = 0.061
x = [-0.567 -0.982], target = -0.302, predicted = -0.281
x = [ 1.340  0.203], target = 0.949, predicted = 0.918
```

EXP - 3

Code:

```
import numpy as np

class Perceptron:
    def __init__(self, input_size, lr=0.1, epochs=20):
        self.weights = np.zeros(input_size)        self.bias =
0.0
        self.lr = lr
        self.epochs = epochs

    def activation(self, x):
        # Step (threshold) activation function
        return 1 if x >= 0 else 0

    def predict(self, x):
        z = np.dot(self.weights, x) + self.bias
        return self.activation(z)

    def train(self, X, y):
        for epoch in range(self.epochs):
            total_error = 0
            for xi, target in zip(X, y):
                pred = self.predict(xi)
                error = target - pred
                # Perceptron learning rule
                self.weights += self.lr * error * xi
                self.bias += self.lr * error
                total_error += abs(error)
if total_error == 0:
                print(f"Converged at epoch {epoch + 1}")
                break

    def test(self, X, y):
        print(f"{'Input':>10} {'Target':>8} {'Predicted':>10}")
        for xi, target in zip(X, y):
            pred = self.predict(xi)
            print(f"{str(xi):>10} {target:>8} {pred:>10}")
```

```
# ---------------- Truth tables ----------------
X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])

gates = {
   "AND": np.array([0, 0, 0, 1]),
   "OR":  np.array([0, 1, 1, 1]),
   "NAND": np.array([1, 1, 1, 0]),
}

if __name__ == "__main__":
   for gate_name, y in gates.items():
      print(f"\n===== Training Perceptron for {gate_name} gate =====")
      p = Perceptron(input_size=2, lr=0.1, epochs=20)
      p.train(X, y)
      print(f"Final weights: {p.weights}, bias: {p.bias:.2f}")
      p.test(X, y)
```

Output:

```
===== Training Perceptron for AND gate =====
Converged at epoch 5
Final weights: [0.2 0.1], bias: -0.20
    Input   Target  Predicted
   [0 0]       0        0
   [0 1]       0        0
   [1 0]       0        0
   [1 1]       1        1

===== Training Perceptron for OR gate =====
Converged at epoch 4
Final weights: [0.1 0.1], bias: -0.10
    Input   Target  Predicted
   [0 0]       0        0
   [0 1]       1        1
   [1 0]       1        1
   [1 1]       1        1
```

```
===== Training Perceptron for NAND gate =====
Converged at epoch 6
Final weights: [-0.2 -0.1], bias: 0.20      Input
Target  Predicted
   [0 0]       1         1
   [0 1]       1         1
   [1 0]       1         1
   [1 1]       0         0
```

EXP - 4

Code:

```
import random

# ---------------- Problem setup ----------------
CITY_COUNT   = 10
POP_SIZE     = 60
GENERATIONS  = 300
MUTATION_RATE = 0.02
ELITE_SIZE   = 5

random.seed(42)
cities = [(random.randint(0, 200), random.randint(0, 200))
        for _ in range(CITY_COUNT)]

def distance(a, b):
   return ((a[0] - b[0]) ** 2 + (a[1] - b[1]) ** 2) ** 0.5

def route_length(route):
   return sum(distance(cities[route[i]], cities[route[(i + 1) % len(route)]])
            for i in range(len(route)))

def fitness(route):
   return 1 / (route_length(route) + 1e-6)

# ---------------- GA operators ----------------
def create_route():
   r = list(range(CITY_COUNT))
   random.shuffle(r)
   return r

def initial_population():
   return [create_route() for _ in range(POP_SIZE)]
 def rank_population(population):

   return sorted(population, key=fitness, reverse=True)

def selection(ranked_pop):
```

```
    elites = ranked_pop[:ELITE_SIZE]
    tournament = random.sample(ranked_pop, 5)
    winner = max(tournament, key=fitness)

    return elites, winner

def ordered_crossover(parent1, parent2):
    a, b = sorted(random.sample(range(CITY_COUNT), 2))
    child = [None] * CITY_COUNT
    child[a:b] = parent1[a:b]
    fill = [g for g in parent2 if g not in child]
    idx = 0
    for i in range(CITY_COUNT):
        if child[i] is None:
            child[i] = fill[idx]
            idx += 1
    return child

def mutate(route):
    for i in range(len(route)):
        if random.random() < MUTATION_RATE:            j
= random.randint(0, len(route) - 1)
            route[i], route[j] = route[j], route[i]
    return route

def next_generation(population):
    ranked = rank_population(population)
    elites, _ = selection(ranked)
    children = elites[:]
    while len(children) < POP_SIZE:
        p1 = random.choice(ranked[:20])
        p2 = random.choice(ranked[:20])        child =
ordered_crossover(p1, p2)
        children.append(mutate(child))
    return children

# ---------- Main GA loop ----------
def genetic_algorithm():
```

```
    population = initial_population()
    best_history = []
    for gen in range(GENERATIONS):
        population = next_generation(population)
        best_route = rank_population(population)[0]
best_history.append(route_length(best_route))
        if gen % 50 == 0:
            print(f"Generation {gen}: best distance =
{route_length(best_route):.2f}")
    best_route = rank_population(population)[0]
    return best_route, route_length(best_route), best_history
 if __name__ == "__main__":

    best_route, best_distance, history = genetic_algorithm()
    print("\nBest route found:", best_route)
    print(f"Best distance: {best_distance:.2f}")
```

Output:

```
Generation 0:   best distance = 1084.32
Generation 50:  best distance = 742.15
Generation 100: best distance = 651.78
Generation 150: best distance = 611.40
Generation 200: best distance = 598.02
Generation 250: best distance = 591.67


Best route found: [3, 7, 1, 9, 4, 0, 6, 2, 8, 5]
Best distance: 588.94
```

EXP - 5

Code:

```
# ---------------- Fuzzy Set Operations ----------------
# Fuzzy sets represented as Python dictionaries:
# key   -> element
# value -> degree of membership (in [0, 1])

A = {"x1": 0.2, "x2": 0.5, "x3": 0.7, "x4": 1.0, "x5": 0.0}
B = {"x1": 0.6, "x2": 0.3, "x3": 0.9, "x4": 0.4, "x5": 0.8}

def union(A, B):
    """(A U B)(x) = max(A(x), B(x))"""     return {x:
max(A[x], B[x]) for x in A}


def intersection(A, B):
    """(A n B)(x) = min(A(x), B(x))"""
    return {x: min(A[x], B[x]) for x in A}

def complement(A):
    """A'(x) = 1 - A(x)"""
    return {x: round(1 - A[x], 2) for x in A}

def difference(A, B):
    """(A - B)(x) = min(A(x), 1 - B(x))"""
    return {x: round(min(A[x], 1 - B[x]), 2) for x in A}

def algebraic_sum(A, B):
    """(A +alg B)(x) = A(x) + B(x) - A(x)*B(x)"""
    return {x: round(A[x] + B[x] - A[x] * B[x], 2) for x in A}

def algebraic_product(A, B):
    """(A *alg B)(x) = A(x) * B(x)"""
    return {x: round(A[x] * B[x], 2) for x in A}
 def is_subset(A, B):

    """A subseteq B if A(x) <= B(x) for all x"""
    return all(A[x] <= B[x] for x in A)
```

```
def cardinality(A):
    """Sum of membership values (scalar cardinality)"""
    return round(sum(A.values()), 2)



if __name__ == "__main__":
    print("Fuzzy Set A:", A)
    print("Fuzzy Set B:", B)
    print()
    print("Union (A U B)         :", union(A, B))
    print("Intersection (A n B)   :", intersection(A, B))
    print("Complement (A')       :", complement(A))
    print("Complement (B')       :", complement(B))
    print("Difference (A - B)    :", difference(A, B))
    print("Algebraic Sum         :", algebraic_sum(A, B))
    print("Algebraic Product     :", algebraic_product(A, B))
    print("Is A subset of B?     :", is_subset(A, B))
    print("Cardinality of A      :", cardinality(A))
    print("Cardinality of B      :", cardinality(B))
```

Output:

```
Fuzzy Set A: {'x1': 0.2, 'x2': 0.5, 'x3': 0.7, 'x4': 1.0, 'x5': 0.0}
Fuzzy Set B: {'x1': 0.6, 'x2': 0.3, 'x3': 0.9, 'x4': 0.4, 'x5': 0.8}

Union (A U B)         : {'x1': 0.6, 'x2': 0.5, 'x3': 0.9, 'x4': 1.0, 'x5': 0.8} Intersection (A n B)   :
{'x1': 0.2, 'x2': 0.3, 'x3': 0.7, 'x4': 0.4, 'x5': 0.0}
Complement (A')       : {'x1': 0.8, 'x2': 0.5, 'x3': 0.3, 'x4': 0.0, 'x5': 1.0}
Complement (B')       : {'x1': 0.4, 'x2': 0.7, 'x3': 0.1, 'x4': 0.6, 'x5': 0.2}
Difference (A - B)    : {'x1': 0.2, 'x2': 0.5, 'x3': 0.1, 'x4': 0.6, 'x5': 0.0}
Algebraic Sum         : {'x1': 0.68, 'x2': 0.65, 'x3': 0.97, 'x4': 1.0, 'x5': 0.8}
Algebraic Product     : {'x1': 0.12, 'x2': 0.15, 'x3': 0.63, 'x4': 0.4, 'x5': 0.0}
Is A subset of B?     : False
Cardinality of A      : 2.4
Cardinality of B      : 3.0
```

EXP – 6

NAME: PANKAJ RANA
DIV: B
ROLL NO: 29

Case Study on Applications of Hybrid Computing

1. Introduction

Hybrid computing is an approach that combines two or more computing techniques, architectures, or technologies to solve complex problems efficiently. It combines the strengths of different systems to improve performance, accuracy, flexibility, and reliability.

In artificial intelligence, hybrid computing may combine soft computing techniques such as fuzzy logic and artificial neural networks. It can also combine CPU and GPU processing or integrate classical computing with quantum computing.

This case study focuses on the application of hybrid computing in weather forecasting, where different computing techniques work together to predict weather conditions more effectively.

2. Problem Statement

Weather forecasting is a complex task because atmospheric conditions continuously change and depend on several factors, such as temperature, humidity, wind speed, air pressure, and rainfall.

Traditional forecasting methods may face difficulties in predicting sudden weather changes using large and complex datasets. Therefore, a hybrid computing system is used to combine numerical weather prediction models with artificial intelligence techniques to improve forecasting and prediction accuracy.

3. Techniques Used

1. Numerical Weather Prediction (NWP): Uses mathematical equations and atmospheric data to simulate weather conditions.

2. Artificial Neural Networks (ANN): Analyze historical weather data and identify patterns to predict future conditions.

3. Fuzzy Logic: Handles uncertainty in weather conditions and helps classify situations such as low, moderate, or high rainfall risk.

4. High-Performance Computing (HPC): Uses powerful computing systems to process large volumes of satellite, radar, and atmospheric data.

4. Working of the Hybrid Computing System

The system works through the following steps:

1. Data Collection: Weather data is collected from satellites, weather stations, radar systems, and sensors.
2. Data Processing: The collected data is cleaned and organized for analysis.
3. Numerical Simulation: NWP models calculate atmospheric conditions using mathematical equations.
4. AI-Based Prediction: Neural networks analyze historical and current weather data to identify patterns.
5. Uncertainty Analysis: Fuzzy logic helps interpret uncertain conditions and classify potential weather risks.
6. Hybrid Integration: The outputs of numerical models and AI techniques are combined to generate improved predictions.
7. Forecast Generation: The system produces forecasts for temperature, rainfall, storms, and other weather conditions.
8. Decision-Making: Weather authorities use the forecast to issue warnings and support public safety.

5. Applications

Hybrid computing has several applications in weather forecasting and other fields:

- Weather Forecasting: Predicts temperature, rainfall, and wind conditions.
- Disaster Management: Supports early warnings for floods, cyclones, and thunderstorms.
- Healthcare: Combines AI models and medical information to support disease diagnosis.
- Autonomous Vehicles: Integrates sensor data and AI for navigation and obstacle detection.
- Industrial Automation: Combines control systems and AI to improve manufacturing efficiency.
- Financial Analysis: Integrates statistical models and machine learning to analyze financial trends.

6. Advantages

- Combines the strengths of multiple computing techniques.
- Can improve prediction accuracy and reliability.
- Processes large and complex datasets efficiently.
- Handles uncertain and changing environmental conditions.
- Supports faster and better decision-making.

7. Limitations

- System design and integration can be complex.
- Requires large amounts of reliable data.
- Implementation and maintenance may be expensive.

- Different techniques may produce conflicting results.
- Requires skilled professionals to develop and manage the system.

8. Conclusion

Hybrid computing is a powerful approach that combines different computing techniques to solve complex real-world problems. In weather forecasting, it integrates numerical weather prediction, artificial intelligence, fuzzy logic, and high-performance computing to analyze atmospheric data and generate forecasts.

Although hybrid computing requires careful integration, reliable data, and significant computational resources, it can improve forecasting capabilities and support early warnings for severe weather events. Therefore, hybrid computing plays an important role in modern intelligent systems and scientific applications.